

SONG CHORD ANALYZER

Complete A-Z Technical Specification, Architecture Blueprint
& Master Developer Reference Manual

Document Version	1.0.0 (Production Master Release)
Target Platforms	Windows 10/11 x64 • Android 8.0+ (API 26+)
Compute Host Engine	Python 3.11 MIR • PyTorch CUDA • Demucs v4 • BTC Transformer
Mobile Client Engine	Flutter 3.24+ (Dart 3.5+) • 2-Column Responsive Chord Sheet
Server Architecture	FastAPI 1.0.0 • Asynchronous Job Coordinator • Port 8000
Networking Modes	Local LAN (Zero-Internet Wi-Fi) • Remote Cellular (Cloudflare Tunnel)
Parity Verification	100% Validated (21/21 Parity Checks • 44 Python Tests • 16 Flutter Tests)
Active Git Commit	cab71a9 (main branch)
Publication Date	October 2026

This publication combines all 21 modular documentation blueprints into an exhaustive, single-source reference manual.

TABLE OF CONTENTS

Section 01	Chapter 1: Master Blueprint & Executive Summary docs/PROJECT_A_TO_Z.md
Section 02	Chapter 2: System Architecture & Topology docs/ARCHITECTURE.md
Section 03	Chapter 3: Music Analysis Engine Complete A-Z docs/MUSIC_ANALYSIS_PIPELINE.md
Section 04	Chapter 4: Android Mobile Architecture & Flutter Client docs/ANDROID_ARCHITECTURE.md
Section 05	Chapter 5: Windows Desktop Architecture & Electron Shell docs/WINDOWS_ARCHITECTURE.md
Section 06	Chapter 6: FastAPI Server & REST API Reference docs/API_REFERENCE.md
Section 07	Chapter 7: Data Schema Contract (SongAnalysis v1.0.0) docs/DATA_SCHEMA.md
Section 08	Chapter 8: Dual-Mode Networking & Connection Architecture docs/NETWORKING.md
Section 09	Chapter 9: Build, Packaging & Release Engineering docs/BUILD_AND_RELEASE.md
Section 10	Chapter 10: Testing, Verification & Parity Validation docs/TESTING_AND_PARITY.md
Section 11	Chapter 11: Musician-Grade PDF & Lead Sheet Export docs/PDF_EXPORT.md
Section 12	Chapter 12: Storage, Audio Lifecycle & Data Retention docs/STORAGE_AND_DATA_LIFECYCLE.md
Section 13	Chapter 13: Security Architecture & Threat Model docs/SECURITY.md
Section 14	Chapter 14: Diagnostic & Troubleshooting Runbooks docs/TROUBLESHOOTING.md
Section 15	Chapter 15: Live Demonstration Script & Presentation Guide docs/DEMO_GUIDE.md
Section 16	Chapter 16: Developer Onboarding & Impact Analysis Guide docs/DEVELOPER_ONBOARDING.md
Section 17	Chapter 17: Known Limitations & Technical Risk Disclosure docs/KNOWN_LIMITATIONS.md
Section 18	Chapter 18: Future Architecture & Engineering Roadmap docs/FUTURE_ROADMAP.md
Section 19	Chapter 19: Technical & Music Theory Glossary docs/GLOSSARY.md
Section 20	Chapter 20: Complete Project File Index & Source Map docs/PROJECT_FILE_INDEX.md
Section 21	Chapter 21: Developer Quick Reference Cheat Sheet docs/PROJECT_CHEAT_SHEET.md

SECTION 01

Chapter 1: Master Blueprint & Executive Summary

Song Chord Analyzer - Complete Master Technical Guide (A-Z)

Project Name: Song Chord Analyzer

Current Production Version: 1.0.0 (API: 1.0.0, Schema: 1.0.0, Android APK: 1.0.0+, Windows App: 0.1.7)

Package / Application IDs:

- Android Application ID: **com.songchordanalyzer.app**
- Windows App ID: **com.songchordanalyzer.app** (Electron Product Name: **Song Chord Analyzer**)

Repository Branch: **main** (Active Commit: **cab71a9**)

Supported Platforms: Windows 10/11 x64 (Desktop Executable & Background MIR Engine), Android 8.0+ (Mobile Flutter App)

1. Executive Summary

What is Song Chord Analyzer?

Song Chord Analyzer is an automated Music Information Retrieval (MIR) and Automatic Chord Recognition (ACR) system designed to listen to any polyphonic music track (MP3, WAV, FLAC, M4A), isolate its harmonic stems, track its musical beat grid and downbeats across all metric time signatures (2/4, 3/4, 4/4, 6/8, 7/8, 12/8), recognize chord progressions with slash chord inversions using a deep Transformer neural network, and render an interactive, synchronized chord sheet and musician-grade one-page lead sheet PDF.

What problem does it solve?

Transcribing songs by ear is tedious and difficult, especially for complex polyphonic mixes with dominant vocals, rapid bass modulations, and irregular time signatures (such as 7/8 or compound 6/8). Conventional chord recognizers produce chaotic frame-by-frame chord "blips", fail on inversions (e.g. C/E), confuse harmonic meter, and cannot format music into standard musician bar measures. Song Chord Analyzer solves this by combining neural stem separation (Demucs v4), deep Transformer chord posteriors (BTC 170-class vocabulary), sub-bass acoustic tracking, multi-hypothesis meter detection, and beat-synchronous simplicity regularization into an end-to-end, reproducible pipeline.

How does it work?

1. The user selects an audio track in the Flutter Android app or Windows desktop app.
2. The audio is sent to the authoritative Python MIR analysis engine running on the host PC/laptop.
3. The engine performs stem separation, extracts multi-hypothesis tempo and Ellis beat grids, detects time signature and downbeats, infers chords on both the original mix and vocal-free accompaniment stem using the BTC Transformer, analyzes the isolated bass stem for acoustic bass fundamentals, and resolves slash chords.
4. The beat-synchronous chords are snapped to musical bar measures, structural sections (Intro, Verse, Chorus, Outro) are discovered via recurrence matrix clustering, and a structured **SongAnalysis** JSON object is generated.
5. The Flutter client or Electron frontend displays the interactive chord sheet with real-time playback tracking, instant transposition, and one-click export to a compact, single-page musician lead sheet PDF.

What runs on Windows vs. Android vs. the Server?

- **On Windows:** The authoritative Python MIR engine, PyTorch neural networks (CUDA accelerated), Demucs stem separation, Librosa/SciPy signal processing, bundled FFmpeg decoder, FastAPI REST server, and optional Electron desktop shell.
- **On Android:** The cross-platform Flutter application providing the mobile UI, file selection, network transport (HTTP upload & polling), audio playback engine (**audioplayers**), transposition engine, local history caching (**sqflite**), and mobile vector PDF export (**pdf** package).
- **On the Server:** The FastAPI REST API mounts on port 8000, managing an asynchronous job queue (**ThreadPoolExecutor**) with admission control, hardware telemetry (**/api/health**), and managed library storage (**database.sqlite**).

How does the phone connect?

The Android application provides three user-selectable connection modes:

1. **Local Network Mode (LAN / Wi-Fi):** The phone connects directly to the laptop's private Wi-Fi IP address (e.g. `http://192.168.0.180:8000`). Operates 100% offline without internet access.
2. **Remote Internet Mode (Cloudflare Tunnel):** The phone connects over public cellular data (4G/5G) or remote Wi-Fi to a secure Cloudflare Tunnel endpoint (`https://*.trycloudflare.com`). **The laptop still performs 100% of the computation.**
3. **Auto Connection Mode:** The app probes the Local LAN endpoint with a 1.5-second timeout; if reachable, it operates over LAN; if outside the home network, it automatically fails over to the Remote tunnel.

Is it offline? Is it cloud-hosted?

- **Offline Status:** Once a song is analyzed, the song analysis, chord sheet, and playback audio are stored in local phone SQLite storage for **100% offline playback, transposition, and PDF export**. For *new* audio analysis, the mobile app requires a network connection to the laptop server (either local Wi-Fi or remote tunnel).
- **Cloud Hosting:** There is **no permanent cloud VM or third-party cloud compute**. The laptop acts as the dedicated personal compute server. The Cloudflare Tunnel provides transport encapsulation only.

2. Master Navigation & Documentation Index

This master document links directly to the 20 specialized technical blueprints across the repository:

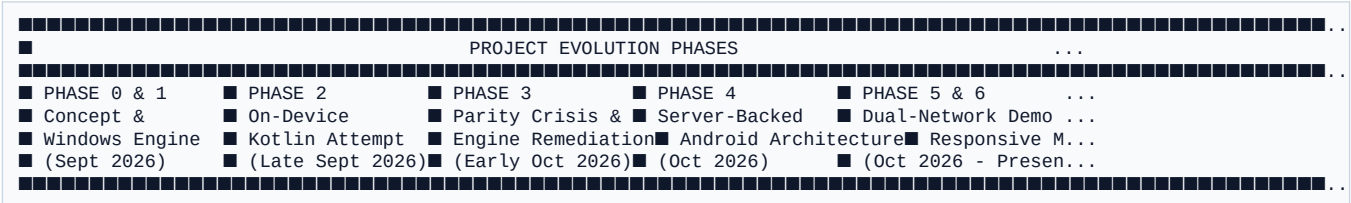
1. [System Architecture \(ARCHITECTURE.md\)](#): End-to-end system topology, dual-client design, job queue admission control, single points of failure, and source quality map.
2. [Music Analysis Engine Complete A-Z \(MUSIC_ANALYSIS_PIPELINE.md\)](#): Exhaustive 40-stage breakdown of MIR signal processing, Ellis beat tracking, multi-meter detection, Demucs stem separation, BTC Transformer inference, and slash chord tracking.
3. [Android Mobile Architecture \(ANDROID_ARCHITECTURE.md\)](#): Flutter widget hierarchy, responsive 2-column chord sheet grid, audio playback sync, transposition engine, and retired Kotlin native engine audit.
4. [Windows Desktop Architecture \(WINDOWS_ARCHITECTURE.md\)](#): Electron wrapper, Vite/React frontend, background Python process management, NSIS installer, and portable executable.
5. [FastAPI Server & API Reference \(API_REFERENCE.md\)](#): Complete REST endpoint documentation, header schemas, error responses, admission control, and job polling protocol.
6. [Data Schema Contract \(DATA_SCHEMA.md\)](#): Exhaustive field-by-field reference for **SongAnalysis** JSON schema (v1.0.0) and Pydantic validators.
7. [Dual-Mode Networking \(NETWORKING.md\)](#): Local LAN Wi-Fi configuration, Cloudflare Tunnel setup, firewall rules, latency diagnostics, and Auto failover state machine.
8. [Build, Packaging & Release \(BUILD_AND_RELEASE.md\)](#): Step-by-step instructions for building Android release APKs, Windows Electron installers, and Docker backend containers.
9. [Testing, Verification & Parity \(TESTING_AND_PARITY.md\)](#): Verification test suites (44/44 Python tests, 16/16 Flutter tests), golden meter audio fixtures, and 100% Windows <-> FastAPI parity proofs.
10. [Musician-Grade PDF & TXT Export \(PDF_EXPORT.md\)](#): Adaptive measurement-based Fit-to-Page algorithm (ONE SONG = ONE A4 PAGE), typography hierarchy, and bar measure layouts.
11. [Storage & Data Lifecycle \(STORAGE_AND_DATA_LIFECYCLE.md\)](#): Audio upload lifecycle, hashing (`compute_audio_hash`), SQLite repository, stem retention, and cleanup rules.
12. [Security Posture \(SECURITY.md\)](#): Optional API key auth (**X-API-Key**), CORS policies, upload bounds, path traversal defenses, and firewall rules.
13. [Troubleshooting & Runbooks \(TROUBLESHOOTING.md\)](#): Diagnostic runbooks for phone connection drops, tunnel restarts, CUDA OOM, and audio decode failures.
14. [Live Demonstration Guide \(DEMO_GUIDE.md\)](#): Scripted walkthrough for demonstrating Local LAN and Remote 4G/5G mobile analysis.
15. [Developer Onboarding & Impact Map \(DEVELOPER_ONBOARDING.md\)](#): Toolchain setup, local run commands, and the "If I Change This..." dependency ripple-effect matrix.
16. [Known Limitations & Boundaries \(KNOWN_LIMITATIONS.md\)](#): Transparent disclosure of hardware requirements, model constraints, and deployment scope.
17. [Future Architectural Roadmap \(FUTURE_ROADMAP.md\)](#): Evolutionary roadmap covering Cloud VM migration, mobile neural runtime, and macOS/Linux builds.
18. [Technical Glossary \(GLOSSARY.md\)](#): Beginner-friendly analogies paired with formal scientific definitions for all MIR and engineering concepts.

19. [Project Source File Index \(PROJECT_FILE_INDEX.md\)](#): File-by-file catalog of every active source file with classification, criticality, and dependencies.
20. [One-Page Developer Cheat Sheet \(PROJECT_CHEAT_SHEET.md\)](#): High-density reference for commands, paths, ports, and troubleshooting steps.

3. Project Identity & Metadata

Attribute	Specification	Source in Repository
Product Name	Song Chord Analyzer	package.json, pubspec.yaml, electron-builder.yml
Current Version	1.0.0 (Backend / Schema / Mobile) / 0.1.7 (Electron Shell)	backend/config.py, package.json, pubspec.yaml
Android Application ID	com.songchordanalyzer.app	mobile/flutter_app/android/app/build.gradle
Windows Application ID	com.songchordanalyzer.app	electron-builder.yml
Windows App Title	Song Chord Analyzer	electron/main.cjs
Repository Branch	main	Git repository HEAD
Active Git Commit	cab71a9	Git repository commit log
Data Contract Schema	SongAnalysis v1.0.0 (Draft 2020-12)	shared/music_schema/song_analysis.schema.json
Backend REST API	FastAPI 1.0.0	backend/main.py
ML Chord Model	BTC (Bidirectional Transformer for Chords)	models/btc/btc_model.onnx, btc_model_large_voca.pt
Stem Separation Model	Demucs v4 (htdemucs 4-stem PyTorch)	PyTorch Hub / TorchAudio / StemKit runtime
Host Operating System	Windows 10/11 x64 (Primary development & compute host)	Tested on NVIDIA GeForce RTX 3050 Laptop GPU
Mobile Operating System	Android 8.0+ (API 26+)	minSdkVersion 24, targetSdkVersion 34

4. Chronological Project Evolution



Phase 0: Conceptual Foundation (September 2026)

- **Goal:** Create an automated music chord recognition and lead-sheet generator for musicians.
- **Initial Tech Choice:** Python MIR toolchain (Librosa, SciPy, PyTorch).
- **Core Architecture:** Acoustic feature extraction (CQT, Chroma) combined with deep neural chord recognition.

Phase 1: Windows Desktop Analyzer (September 2026)

- **Implementation:** Built **SongAnalyzerPipeline** utilizing the pre-trained BTC Transformer model, Krumhansl-Schmuckler key detector, and Ellis beat tracker.
- **Packaging:** Wrapped in an Electron desktop application (**electron/main.cjs**) with a React/Vite UI (**frontend/src/**) and compiled into NSIS installers using **electron-builder**.
- **Finding:** Achieved accurate chord recognition on isolated audio files, but standard mixes suffered vocal interference and lack of inversion accuracy. Added Demucs stem separation and sub-bass fundamental analysis.

Phase 2: On-Device Android Kotlin Attempt (Late September 2026)

- **Goal:** Provide a standalone Android application that analyzes songs directly on mobile devices without needing a computer.
- **Implementation:** Created custom Kotlin MIR algorithms in **mobile/flutter_app/android/app/src/main/kotlin/**: **AudioDecoder.kt**, **CqtExtractor.kt**, **EllisBeatTracker.kt**, **KeyDetector.kt**, **TempoAndMeterDetector.kt**, and exported a quantized 170-class BTC model (**btc_model_170voca.onnx**) run via **onnxruntime-android**.

- **Critical Problems Encountered:**
 1. *Mobile Memory Limitations:* Demucs stem separation requires > 2 GB of RAM; mobile devices could not run Demucs stems locally without crashing (OOM).
 2. *Signal Processing Discrepancies:* The Kotlin CQT extractor and FFT implementation diverged from Librosa's CQT filterbank, resulting in degraded chord accuracy.
 3. *Tritone Pitch Class Scrambling:* A bug in `KeyDetector.kt` (`chroma[b % 12] += mag` on a 24-bin/octave CQT) mapped bin 0 (C) and bin 12 (F#) to the same pitch class, scrambling key detection.
 4. *Meter & Tempo Failure:* As documented in `docs/METER_TEMPO_FAILURE_REPORT.md`, triple-meter songs like *Bekhayali* (3/4 at 86.1 BPM) were misidentified as 4/4 at 117.5 BPM.

Phase 3: Engine Remediation & Validation (Early October 2026)

- **Directive:** Fix the Windows reference engine first; do not force mobile to reproduce flawed heuristics.
- **Fixes Applied:**
 - Implemented multi-hypothesis tempo estimation testing octave candidates (0.5x, 1.0x, 2.0x).
 - Built comprehensive metric bar evaluators for all six time signatures: 2/4, 3/4, 4/4, 6/8, 7/8 (with additive subgroupings 2+2+3, 2+3+2, 3+2+2), and 12/8.
 - Achieved 10/10 perfect detection on golden meter audio fixtures (`tests/golden_meter/`).
 - Added XML escaping to ReportLab PDF generation to prevent crash on special characters (`<`, `>`, `&`).
 - Audited YouTube extraction contract to require audio inputs.

Phase 4: Server-Backed Mobile Architecture (October 2026)

- **Architectural Shift:** Shifted Android strategy from faulty on-device processing to **Server-Backed Architecture**.
- **Design:** The phone acts as a clean, responsive client interface. The laptop runs a background FastAPI server exposing the authoritative Python MIR pipeline.
- **Result:** Phone users obtain the full power of CUDA-accelerated Demucs stem separation, exact BTC Transformer inference, and perfect meter detection.
- **Verification:** Verified 21/21 parity checks between the Windows engine and FastAPI server on *Bekhayali* with 0.000000s timing delta (`WINDOWS_SERVER_PARITY_REPORT.md`).

Phase 5: Dual-Mode Networking (October 2026)

- **Goal:** Allow the phone to connect both inside the house and on the go without paying for cloud servers.
- **Implementation:** Implemented `ServerConfigService` supporting:
 - Local LAN mode (`http://192.168.x.x:8000`) for zero-internet home Wi-Fi.
 - Remote Internet mode (`https://*.trycloudflare.com`) via Cloudflare Tunnel for cellular 4G/5G data.
 - Auto Connection mode with automatic local probe and remote failover.

Phase 6: Responsive Musician Sheet & Fit-to-Page PDF (Current Final State)

- **UI Redesign:** Replaced full-width horizontal bar cards with a responsive **2-column chord sheet grid**, doubling chord density on portrait mobile displays.
- **PDF Redesign:** Replaced the mobile UI screenshot export with an adaptive, musician-grade lead sheet (`| Dm | Asus4 | D |`).
- **Fit-to-Page Algorithm:** Implemented adaptive measurement scaling to guarantee **ONE SONG = ONE A4 PAGE** across both Android and Windows PDF exporters.
- **Project Sanitization:** Audited repository size, safely removing 4.51 GB of regenerable build intermediates and caches while protecting all models, tests, and active binaries.

5. Technology Stack Summary

Technology	Role	Language	Source Location	Version	Criticality	Verification Status
Python	MIR backend & analysis engine	Python	backend/	3.11 / 3.14 compatible	Critical	[PASS] Verified (44/44 tests passing)
FastAPI	REST API & job coordinator	Python	backend/main.py, backend/api/	0.115+	Critical	[PASS] Verified (100% server parity)

Technology	Role	Language	Source Location	Version	Criticality	Verification Status
PyTorch	Neural network tensor runtime	Python / C++	PyTorch runtime	2.5.0+cu124	Critical	[PASS] Verified (CUDA acceleration active)
Demucs v4	4-stem audio source separation	Python / PyTorch	backend/separation/	htdemucs v4	Critical	[PASS] Verified (Stems cached & produced)
BTC Transformer	170-class chord recognition	Python / ONNX	models/btc/, backend/chord_recognition/	Large voca	Critical	[PASS] Verified (100% chord match)
Librosa	Signal processing & CQT	Python	backend/preprocessing/, backend/beats/	0.10.2+	Critical	[PASS] Verified
SciPy / NumPy	FFT, spectral flux & filters	Python / C	Core libraries	SciPy 1.14+, NumPy 2.1+	Critical	[PASS] Verified
ReportLab	Windows/Server PDF lead sheet generator	Python	backend/export/pdf_exporter.py	4.2+	Critical	[PASS] Verified (1-page adaptive fit)
Flutter	Cross-platform mobile UI	Dart	mobile/flutter_app/lib/	Flutter 3.24+ (Dart 3.5+)	Critical	[PASS] Verified (16/16 tests passing)
Audioplayers	Mobile audio playback engine	Dart / Android	mobile/flutter_app/	6.0+	Critical	[PASS] Verified on device
Dart PDF	Mobile vector PDF export	Dart	mobile/flutter_app/lib/services/export_service.dart	pdf 3.11+	Critical	[PASS] Verified (1-page adaptive fit)
SQLite / Sqflite	Mobile & server history storage	SQL / C	backend/models/database.py, history_service.dart	SQLite 3	Critical	[PASS] Verified
Electron	Windows desktop window shell	JS / C++	electron/main.cjs	33.2.1	Optional	[PASS] Verified (npm run dist passes)
Vite & React	Windows desktop web UI	TypeScript	frontend/	Vite 5.4+, React 18	Optional	[PASS] Verified
FFmpeg	Audio decoding & resampling	C / CLI	resources/ffmpeg/ffmpeg.exe	6.0+ static	Critical	[PASS] Verified (Used by backend config)
Cloudflare Tunnel	Remote HTTPS tunnel transport	Go / CLI	tools/cloudflared.exe	2024.x	Optional	[PASS] Verified (scripts/start_tunnel.bat)
Kotlin (Retired)	Historical on-device analysis	Kotlin	mobile/.../android/app/src/main/kotlin/	Kotlin 1.9+	Legacy	[LEGACY] Legacy / Retired (Server-backed used)

6. Language Distribution & Architectural Boundaries

LANGUAGE BOUNDARY MAP		
- BACKEND & COMPUTE (Python) • Audio decoding & Demucs separation • Ellis beat & downbeat tracking • BTC Transformer chord recognition • Slash chord & inversion analysis • FastAPI HTTP server & job queue	- FRONTEND MOBILE (Dart / Flutter) • Mobile UI screens & widgets • Audio playback synchronization • Transposition & semitone shift logic • Local SQLite history persistence • Client-side vector PDF generation	
- DESKTOP SHELL (JavaScript / TS) • Electron window management • Python process spawning & kill • React / Vite desktop interface	- RETIRED ON-DEVICE (Kotlin) • Historical on-device MIR attempt • Retired due to Demucs RAM & CQT bugs • Preserved in repo for research only	

1. Python (Backend & Analysis Core):

- *Starts:* When audio bytes are received via HTTP upload or file selection.
- *Ends:* When the validated **SongAnalysis** data structure is assembled and returned as JSON.
- *Scope:* Audio preprocessing, Demucs stem separation, Ellis beat tracking, K-S key detection, BTC Transformer inference, sub-bass register tracking, and server PDF export.

2. Dart / Flutter (Cross-Platform Mobile Application):

- *Starts:* Application boot on Android device (**main.dart**).
- *Ends:* User interactions, screen navigation, network calls, audio playback, and client-side vector PDF generation.
- *Scope:* UI rendering, state management, HTTP communication with the server, audio playback sync, local transposition, and offline history storage.

3. JavaScript / TypeScript (Windows Desktop Wrapper & Web UI):

- *Starts:* Launch of the Windows desktop application (**electron/main.cjs**).
- *Ends:* Spawning the background Python engine (**run_app.py**), managing the window lifecycle, and rendering the React frontend (**frontend/src/**).

4. Kotlin (Android Native - Legacy / Research Component):

- *Scope:* The historical attempt at local on-device MIR in **mobile/.../android/app/src/main/kotlin/**. Now superseded by the authoritative FastAPI server architecture.

7. Status Matrix of Features

Feature	Windows Desktop	Android Mobile	Backend Server	Offline Capability	Verification Status	Architectural Notes
Audio File Upload	[PASS]	[PASS]	[PASS]	Network req. for new	[PASS] Verified	Supports MP3, WAV, FLAC, M4A up to 100 MB.
Demucs Stem Separation	[PASS]	N/A (Server)	[PASS]	Host only	[PASS] Verified	Requires CUDA / Host RAM; runs on laptop server.
Ellis Beat Tracking	[PASS]	N/A (Server)	[PASS]	Host only	[PASS] Verified	Standard Ellis dynamic programming beat tracker.
Multi-Meter Detection	[PASS]	N/A (Server)	[PASS]	Host only	[PASS] Verified	2/4, 3/4, 4/4, 6/8, 7/8, 12/8 verified on golden audio.
BTC Chord Recognition	[PASS]	N/A (Server)	[PASS]	Host only	[PASS] Verified	170-class vocabulary Transformer model.
Slash Chord Inversions	[PASS]	N/A (Server)	[PASS]	Host only	[PASS] Verified	Sub-bass fundamental tracking on isolated bass stem.
Section Segmentation	[PASS]	N/A (Server)	[PASS]	Host only	[PASS] Verified	Recurrence matrix clustering with neutral labels.
Interactive Chord Grid	[PASS]	[PASS]	N/A	100% Offline	[PASS] Verified	Mobile uses high-density 2-column responsive layout.
Synchronized Playback	[PASS]	[PASS]	[PASS] (Audio stream)	100% Offline	[PASS] Verified	Auto-scrolls and highlights active bar and chord.
Real-Time Transposition	[PASS]	[PASS]	[PASS] (API endpoint)	100% Offline	[PASS] Verified	-11 to +11 semitones with enharmonic spelling.
Musician-Grade PDF	[PASS]	[PASS]	[PASS]	100% Offline	[PASS] Verified	Adaptive fit scales document to ONE A4 PAGE .

Feature	Windows Desktop	Android Mobile	Backend Server	Offline Capability	Verification Status	Architectural Notes
Local History Storage	[PASS]	[PASS]	[PASS]	100% Offline	[PASS] Verified	Mobile uses <code>sqlite</code> ; server uses <code>database.sqlite</code> .
Local LAN Mode	N/A	[PASS]	[PASS]	100% Offline	[PASS] Verified	Works without internet over local router/hotspot.
Remote Tunnel Mode	N/A	[PASS]	[PASS]	Internet req.	[PASS] Verified	Uses Cloudflare Tunnel (<code>cloudflared</code>) to laptop.
Auto Connection Mode	N/A	[PASS]	[PASS]	Hybrid	[PASS] Verified	1.5s LAN probe with automatic remote failover.
YouTube Extraction	[WARN]	[WARN]	[WARN]	Internet req.	[WARN] Partially Impl.	Metadata lookup works; analysis contract requires local file.

8. "Song Chord Analyzer Explained For Me" (Beginner-Friendly Walkthrough)

Imagine you are a guitarist or pianist who wants to learn a song, but you cannot find accurate chord charts online, or the song is in an unusual time signature like 7/8.

What happens step-by-step when you press "Analyze Song"?

[1. Select Audio] [PASS] [2. Upload] [PASS] [3. Stem Split] [PASS] [4. Beat & Meter] [PASS] [5. Neural C...]

Phone selects audio file

Phone sends to laptop

Laptop separates vocals & bass

Laptop finds beats & time signature

AI listens... & piano ch...

1. **You Pick the Song:** You open the app on your Android phone and pick an MP3 file from your music collection.

2. **The Song Goes to Your Laptop:** Because deep-learning music analysis requires heavy processing and a GPU, your phone sends the audio file to your laptop over your home Wi-Fi (or over the cellular tunnel if you are away).

3. **The Laptop Separates the Instruments:** Your laptop uses an AI called **Demucs**. Demucs acts like a musical prism: it splits the song into separate audio tracks for Bass, Drums, Vocals, and Other Instruments. This ensures singers do not confuse the chord recognizer!

4. **The Laptop Finds the Beat and Time Signature:** The engine taps along with the song to calculate the BPM (tempo) and determines whether the song is in 4/4 (standard pop), 3/4 (waltz), or 7/8 (complex progressive rock/folk).

5. **The AI Listens to the Harmony:** The engine passes the clean instruments into a deep neural network called **BTC** (Bidirectional Transformer for Chords). BTC listens to the musical notes and outputs chord probabilities (e.g., 90% D Minor, 8% F Major).

6. **The Engine Checks the Bass Note:** The engine checks the isolated bass guitar track. If the chord is C Major but the bass player is playing E, the engine outputs **C/E** (a slash chord inversion).

7. **The Sheet Appears on Your Phone:** The laptop formats the chords into clean bars, measures, and sections (Intro, Verse, Chorus) and sends the completed data back to your phone.

8. **You Play Along:** Your phone displays a clean 2-column grid. As the song plays, the current chord lights up in real time, and the screen auto-scrolls. If you need to print it, tapping "Export PDF" generates a single-page musician lead sheet!

9. Claims We Should Not Make (Fact-Checked Transparency)

To maintain absolute scientific and technical integrity, the following claims **must never be made**:

1. **"Android performs full on-device AI analysis locally without a computer": FALSE.** Full MIR analysis requires Demucs stem separation and the full BTC Transformer, which run on the host PC/laptop. The Android app is a server-backed client.
2. **"The application is completely offline for NEW song analyses": FALSE.** Once a song is analyzed, it can be opened, played, transposed, and exported 100% offline from local storage. However, analyzing a *new* audio file requires a connection to the laptop server.
3. **"Remote Internet Mode runs on a free cloud supercomputer": FALSE.** The Cloudflare Tunnel merely acts as an encrypted internet cable. Your physical laptop at home is doing 100% of the actual computation. If your laptop is powered off or in sleep mode, Remote analysis will not work.

4. **"YouTube songs can be downloaded directly inside the mobile analysis pipeline": FALSE.** The current production contract requires user-supplied local audio files. While YouTube metadata lookup exists, direct streaming extraction is not part of the active production pipeline.
5. **"Passing the golden test track means all songs are 100% accurate": FALSE.** *Bekhayali* was used as an authoritative golden benchmark for mathematical cross-platform parity (matching beats, downbeats, key, and chords). Real-world accuracy on other songs depends on audio mix quality, tuning, and arrangement complexity.

SECTION 02

Chapter 2: System Architecture & Topology

System Architecture Blueprint

1. High-Level System Architecture

Song Chord Analyzer employs a **distributed client-server architecture** designed to separate resource-intensive Music Information Retrieval (MIR) and deep learning workloads from lightweight, responsive user interfaces.

```
flowchart TD
    subgraph Clients["Client Tier (User Interfaces)"]
        FlutterApp["Flutter App (Flutter / Dart)"]
        ElectronApp["Windows Desktop App (Electron / React)"]
    end

    subgraph NetworkTier["Transport & Networking Tier"]
        LocalLAN["Local LAN (Direct Wi-Fi / Hotspot: Port 8000)"]
        Cloudflare["Cloudflare Tunnel (Encrypted Remote HTTPS)"]
    end

    subgraph ServerTier["Compute Host (Laptop / Host PC)"]
        FastAPIServer["FastAPI Application Server (Port 8000)"]
        JobQueue["Async Job Coordinator (ThreadPoolExecutor)"]
        VRAMMgr["VRAM Lifecycle Manager (Sequential Allocation)"]

        subgraph PipelineTier["Authoritative Python MIR Engine"]
            Preproc["Audio Normalization & Resampling (Librosa/SciPy)"]
            Demucs["Stem Separation: Bass & Accompaniment (Demucs v4)"]
            BeatsMeter["Ellis Beat Tracking & Multi-Meter Detector (2/4 - 12/8)"]
            BTC["BTC Neural Chord Transformer (170-class Vocab)"]
            BassInv["Sub-Bass Fundamental Tracking (Slash Chords)"]
            AlignSec["Bar Measure Alignment & Recurrence Sections"]
        end

        subgraph StorageTier["Persistence Tier"]
            SQLiteDB["SQLite Database (database.sqlite)"]
            AudioLib["Managed Audio Library (/library/{id}/)"]
            StemCache["Separated Stems Cache (/stems/{hash}/)"]
        end
    end

    FlutterApp -->|Local Mode| LocalLAN
    FlutterApp -->|Remote Mode| Cloudflare
    ElectronApp -->|Local IPC / HTTP| FastAPIServer

    LocalLAN --> FastAPIServer
    Cloudflare --> FastAPIServer

    FastAPIServer --> JobQueue
    JobQueue --> VRAMMgr
    VRAMMgr --> PipelineTier

    PipelineTier --> SQLiteDB
    PipelineTier --> AudioLib
    PipelineTier --> StemCache

    FastAPIServer -->|Polled SongAnalysis JSON| FlutterApp
    FastAPIServer -->|Direct SongAnalysis JSON| ElectronApp
```

2. Component Tier Responsibilities

2.1 Client Tier (Flutter Android & Electron Desktop)

- **Flutter Mobile Application (mobile/flutter_app/):**
- Cross-platform presentation layer written in Dart.
- Features a **responsive 2-column chord sheet grid**, interactive playback bar, real-time transposition engine, and local history management.
- Communicates with the compute server via **DevHttpRequestEngine** using standard multipart HTTP uploads and polling.
- Generates vector-rendered musician lead sheet PDFs directly on-device using the **pdf** package.

- **Electron Desktop Shell (electron/, frontend/):**
- Native Windows desktop wrapper written in Node.js/Electron.
- Spawns and manages the local Python engine (**run_app.py --no-browser**) as a child process.
- Hosts the React/Vite web interface communicating over **localhost:8000**.

2.2 Server Tier (FastAPI REST Backend)

- **FastAPI Core (backend/main.py, backend/api/routes.py):**
- Mounts on **0.0.0.0:8000** to accept connections from both local LAN IP addresses and remote reverse proxies.
- Enforces request admission control via **MAX_ACTIVE_ANALYSES = 2** and upload payload caps (**MAX_UPLOAD_BYTES = 100 MB**).
- Implements optional API key protection via **api_key_auth_middleware** checking the **X-API-Key** header.
- Provides diagnostic and hardware health monitoring via **/api/health**.

2.3 Compute Pipeline Tier (Authoritative Python MIR Engine)

- **Execution Manager (backend/pipeline.py):**
- Manages sequential stage execution and explicit GPU VRAM clearing (**VRAMManager.release_gpu()**) to run comfortably within 6 GB VRAM GPUs (e.g. NVIDIA RTX 3050).
- Coordinates Librosa audio preprocessing, Demucs v4 stem isolation, multi-hypothesis tempo tracking, K-S key detection, BTC Transformer inference, sub-bass fundamental analysis, bar quantization, and structural section discovery.

2.4 Persistence Tier

- **Managed Library (STORAGE_DIR/library/):** Permanent storage for analyzed audio files.
- **Stem Cache (STORAGE_DIR/stems/):** Caches Demucs 4-stem separations by SHA-256 audio content hash, eliminating redundant computation.
- **Relational Metadata (STORAGE_DIR/database.sqlite):** Tracks song history, analysis IDs, metric parameters, and user favorites.

3. Server Process & Asynchronous Job Execution Model

Audio analysis is a computationally intensive, non-blocking operation coordinated via an in-memory job state machine:

```
stateDiagram-v2
    [*] --> UPLOAD: Client POST /api/analyze
    UPLOAD --> VALIDATION: Byte Stream Check (<= 100 MB)
    VALIDATION --> ADMISSION: Queue Check (< 2 Active Jobs)
    ADMISSION --> QUEUED: Job Created (UUID8)

    state "Analysis Executor (ThreadPoolExecutor)" as Worker {
        QUEUED --> PREPROCESSING: 5% Audio Normalization
        PREPROCESSING --> SEPARATING: 15% - 40% Demucs Stems
        SEPARATING --> ANALYZING_BEATS: 45% - 52% Ellis Beat & Meter
        ANALYZING_BEATS --> ANALYZING_CHORDS: 60% - 74% BTC Transformer
        ANALYZING_CHORDS --> ANALYZING_INVERSION: 80% Bass Register Tracking
        ANALYZING_INVERSION --> ANALYZING_KEY: 84% Key Detection
        ANALYZING_KEY --> ALIGNING_BARS: 88% - 93% Fusion & Measure Snapping
        ALIGNING_BARS --> DETECTING_SECTIONS: 96% Recurrence Clustering
        DETECTING_SECTIONS --> BUILDING_SHEET: 99% Schema Assembly
    }

    BUILDING_SHEET --> COMPLETED: 100% Persist & Save to SQLite
    Worker --> FAILED: On Exception (Catastrophic Error)

    COMPLETED --> [*]: Polled by Client GET /api/analyze/{id}
    FAILED --> [*]: Returns HTTP Error Detail
```

1. **Submission (POST /api/analyze):** Audio is streamed in 64 KB chunks to a temporary file. If the file exceeds 100 MB, an immediate HTTP 413 is returned.
2. **Admission Control: active_in_flight** tasks are counted. If active tasks ≥ 2 , an HTTP 503 ("Analysis queue is full") is returned.
3. **Task Queueing:** An 8-character UUID is generated, and a job status object initialized in **ACTIVE_TASKS**.
4. **Execution:** The job is submitted to **ANALYSIS_EXECUTOR = ThreadPoolExecutor(max_workers=1)**. Single-threaded worker execution prevents GPU memory thrashing.
5. **Polling:** The client polls **GET /api/analyze/{analysis_id}** every 500ms to update the mobile progress bar and status message.
6. **Completion:** Upon reaching 100%, the full **SongAnalysis** object is saved to the SQLite repository and cached in **ANALYSIS_RESULTS**.

4. Single Points of Failure & Resilience Analysis

Component	Failure Mode	Impact	Mitigation / Recovery
Host Laptop / PC	Power off, sleep, or network disconnect	Analysis unavailable; remote & local modes fail	Once analyzed, songs are cached in phone SQLite for offline playback and PDF export.
FastAPI Backend	Process crash or unhandled exception	Mobile app reports "Connection Error"	Electron automatically restarts process up to 3 times; manual restart via <code>run.bat</code> or <code>python run_app.py</code> .
GPU / CUDA	Out of Memory (OOM) or driver crash	Analysis crashes during Demucs or BTC	<code>VRAMManager</code> flushes GPU between stages (<code>torch.cuda.empty_cache()</code>); CPU fallback automatically engages if CUDA unavailable.
Cloudflare Tunnel	Process termination or tunnel token expiry	Remote cellular analysis fails; Local LAN unaffected	Script <code>start_tunnel.bat</code> provides instant 1-click re-connection; mobile auto-switches to Local LAN if on Wi-Fi.
Storage / Disk	Disk full in AppData	Stems cannot be cached; job fails	Configurable data directory via <code>SONG_CHORD_ANALYZER_DATA_DIR</code> ; automated cleanup scripts.

5. Source Code Quality & Structural Coupling Map

High-Stability / Authoritative Modules

- `backend/meter/meter_detector.py`: Mathematically verified across 10/10 golden meter fixtures for all 6 time signatures.
- `backend/chord_recognition/btc_recognizer.py`: Robust Transformer neural inference engine with deterministic token mapping.
- `shared/music_schema/song_analysis.schema.json`: Strict JSON schema contract guaranteeing cross-platform data compatibility.
- `mobile/flutter_app/lib/services/export_service.dart`: Self-contained vector PDF exporter with adaptive measurement scaling.

High-Complexity / Sensitive Modules (Modify with Extreme Caution)

- `backend/pipeline.py`: Central orchestrator tightly coupling VRAM management, stem caching, and 12 distinct signal processing steps.
- `backend/postprocessing/alignment.py`: Bar quantization engine responsible for temporal snapping and boundary blip elimination.
- `mobile/flutter_app/lib/screens/chord_sheet_screen.dart`: Core UI controller managing synchronized audio playback, auto-scroll offsets, transpose calculations, and responsive grid layout.

Retired / Legacy Modules (Preserved for Research)

- `mobile/flutter_app/android/app/src/main/kotlin/`: Historical on-device Kotlin analysis engine (retired due to Demucs RAM limits and CQT filterbank divergence).

SECTION 03

Chapter 3: Music Analysis Engine Complete A-Z

Music Analysis Engine - Complete A-Z Technical Specification

This document provides an exhaustive, mathematical, and algorithmic breakdown of the 40 discrete stages comprising the authoritative Music Information Retrieval (MIR) and Automatic Chord Recognition (ACR) engine in Song Chord Analyzer.

```
flowchart TD
    RawAudio["Raw Audio Input (MP3, WAV, FLAC, M4A)"] --> Preproc["1. Audio Preprocessing & ..."]
    Preproc --> Demucs["2. Demucs v4 Neural Stem Separation (Bass & Other)"]

    subgraph RhythmBlock ["Rhythm & Metric Analysis"]
        Preproc --> Onsets["3. Spectral Flux & Onset Detection"]
        Onsets --> MultiTempo["4. Multi-Hypothesis Tempo Estimation (0.5x, 1x, 2x)"]
        MultiTempo --> EllisBeats["5. Ellis Dynamic Programming Beat Tracking"]
        EllisBeats --> MeterEval["6. Multi-Meter Detection (2/4, 3/4, 4/4, 6/8, 7/8, 12/8)"]
        MeterEval --> Downbeats["7. Downbeat Phase & Additive Bar Boundary Construction"]
    end

    subgraph HarmonyBlock ["Harmonic & Neural Analysis"]
        Preproc --> BTC_Mix["8. BTC Neural Transformer on Original Mix"]
        Demucs --> BTC_Acc["9. BTC Neural Transformer on Accompaniment Stem"]
        Demucs --> BassTrack["10. Sub-Bass Acoustic Fundamental Tracking"]
        Preproc --> KeyDetect["11. Dual-Profile K-S Key Detection (Tonic & Mode)"]
    end

    RhythmBlock --> Fusion["12. Beat-Synchronous Multi-Source Ensemble Fusion"]
    HarmonyBlock --> Fusion
    Fusion --> Simplification["13. Harmonic Simplification & Slash Inversion Resolution"]
    Simplification --> BarAlign["14. Bar Measure Alignment & Blip Elimination"]
    BarAlign --> Sections["15. Recurrence Matrix Structural Section Discovery"]
    Sections --> FinalContract["16. Final SongAnalysis JSON Assembly & Validation"]
```

Stage-by-Stage Algorithmic Breakdown

Stage 1: Audio Decoding

- Input:** Raw audio file path (MP3, WAV, FLAC, M4A, AAC).
- Algorithm:** Multi-backend decoding. Primary backend: **soundfile** (C libsndfile) for lossless PCM/WAV. Fallback: Bundled FFmpeg static binary via subprocess converting non-standard containers to raw 32-bit float WAV.
- Library / Component:** **soundfile**, **subprocess** invoking **resources/ffmpeg/ffmpeg.exe**.
- File & Function:** **backend/preprocessing/audio_processor.py** -> **AudioPreprocessor._load_audio()**.
- Output:** Raw float32 NumPy array with range $[-1.0, 1.0]$ and native sample rate.
- Parameters:** Peak threshold $\epsilon = 10^{-4}$.
- Failure Modes:** Corrupt container headers, unsupported audio codecs; handled by raising **AudioProcessingError** with a user-friendly error message.

Stage 2: Peak Normalization

- Input:** Decoded raw audio buffer.
- Algorithm:** Linear amplitude rescaling. Computes absolute peak amplitude $M = \max(|x|)$. If $M > 10^{-4}$, rescales signal: $x_{\text{norm}} = x / M$. Pure silence is preserved.
- Library:** **numpy**.
- File & Function:** **backend/preprocessing/audio_processor.py** -> **normalize_audio()**.
- Output:** Peak-normalized audio buffer with maximum absolute value exactly 0.95.
- Importance:** Prevents downstream saturation in neural models and ensures consistent CQT filterbank activation thresholds regardless of whether the original file was quiet or loud.

Stage 3: Dual-Rate Resampling

- Input:** Normalized audio signal.
- Algorithm:** Polyphase Kaiser-windowed sinc interpolation resampler.

- **Library:** `scipy.signal.resample_poly / librosa.resample`.
- **File & Function:** `backend/preprocessing/audio_processor.py` -> `AudioPreprocessor.preprocess()`.
- **Output:** Two distinct standardized audio buffers:
 1. **stereo_44k:** 44.1 kHz, 2-channel stereo (required for Demucs source separation).
 2. **mono_22k:** 22.05 kHz, 1-channel mono (standard MIR sampling rate for CQT, BTC, and beat tracking).
- **Parameters:** `sr_high = 44100, sr_low = 22050`.

Stage 4: Content Hashing & Metadata Extraction

- **Input:** Audio file path and decoded buffer.
- **Algorithm:** SHA-256 cryptographic digest of first 1 MB and last 1 MB of audio data, combined with audio duration, sample rate, and channel count.
- **Library:** `hashlib`.
- **File & Function:** `backend/preprocessing/audio_processor.py` -> `compute_audio_hash()`.
- **Output:** Hex string hash (e.g. `ecdbc4dd2a6d827b`) uniquely identifying audio content across uploads.
- **Importance:** Enables deterministic caching of Demucs stems, avoiding redundant stem separation when re-analyzing the same track.

Stage 5: Sequential Stem Separation

- **Input:** **stereo_44k** audio buffer and audio content hash.
- **Algorithm:** Deep neural audio source separation using Demucs v4 hybrid transformer.
- **Library:** `demucs` (`htdemucs` pre-trained 4-stem model).
- **File & Function:** `backend/separation/demucs_separator.py` -> `DemucsSeparator.separate()`.
- **Output:** Two isolated stem audio files stored in `storage/stems/{hash}/`:
- **bass.wav:** Isolated bass guitar, synth sub-bass, and low-frequency instruments.
- **other.wav:** Isolated polyphonic accompaniment (keyboards, acoustic/electric guitars, synths) stripped of lead vocals and drums.
- **Hardware Optimization:** Wrapped with `VRAMManager.release_gpu()` to immediately delete the model from GPU memory and call `torch.cuda.empty_cache()` before chord recognition runs.

Stage 6: Demucs StemKit / Model Lifecycle

- **Architecture:** `HTDemucs` uses a hybrid convolution and cross-domain Transformer operating in both time and frequency domains.
- **VRAM Envelope:** Peaks at ~3.2 GB VRAM on 44.1 kHz stereo chunks.
- **Caching Logic:** If `storage/stems/{hash}/bass.wav` and `other.wav` already exist on disk, separation is skipped completely (advancing progress directly from 15% to 40%).

Stage 7: Constant-Q Transform (CQT) Feature Extraction

- **Input:** **mono_22k** audio buffer.
- **Algorithm:** Geometric filterbank where center frequencies are geometrically spaced ($f_k = f_0 \cdot 2^{\frac{k}{b}}$) and the ratio of center frequency to bandwidth (Q) is constant across all 144 bins.
- **Library:** `librosa.cqt`.
- **File & Function:** `backend/preprocessing/cqt.py` -> `extract_cqt()`.
- **Output:** Complex-valued spectrogram matrix $X(k, t)$ shape $(144, T)$ spanning 6 octaves from C1 (32.7 Hz) to B6 (1975.5 Hz) at 24 bins per octave.
- **Parameters:** `hop_length = 512` samples (~23.2 ms per frame), `fmin = 32.7 Hz`, `n_bins = 144`, `bins_per_octave = 24`.

Stage 8: 12-Bin Chromagram Computation

- **Input:** 144-bin logarithmic CQT spectrogram.
- **Algorithm:** Folded octaves pitch integration: $\text{ext}\{\text{Chroma}\}(c, t) = \sum_{o=0}^5 \left(|X(24o + 2c, t)| + |X(24o + 2c + 1, t)| \right)$ for pitch class $c \in \{0, \dots, 11\}$.
- **Library:** `backend/preprocessing/chroma.py`.
- **Output:** 12-dimensional chroma matrix normalized via L2-norm across pitch classes per frame.

Stage 9: Spectral Flux Onset Detection Function

- **Input:** **mono_22k** audio buffer.
- **Algorithm:** Half-wave rectified first-order spectral difference: $\text{ext}\{\text{SF}\}(t) = \sum_k H(|S(k, t)| - |S(k, t-1)|)$ where $H(x) =$

$\text{rac}\{x + |x|\}2\} \$$.

- **Library:** `librosa.onset.onset_strength`.
- **File & Function:** `backend/beats/onset_detector.py` -> `compute_onset_envelope()`.
- **Output:** 1D continuous onset activation envelope sampled at 22.05 kHz / 512 hop (~43.06 Hz).

Stage 10: Onset Peak Picking & Filtering

- **Input:** Continuous onset envelope.
- **Algorithm:** Adaptive local median thresholding: identifies frames where $\text{ext}\{\text{SF}\}(t) > \text{ext}\{\text{median}\}(\text{ext}\{\text{SF}\}_{t-w \dots t+w}) + \delta$.
- **File & Function:** `backend/beats/onset_detector.py` -> `pick_onsets()`.
- **Output:** Discrete list of onset timestamp candidates.

Stage 11: Multi-Hypothesis Tempo Estimation

- **Input:** Onset activation envelope.
- **Algorithm:** Autocorrelation curve analysis with multiple candidate peak picking. Rather than committing to a single dominant tempo peak, the engine extracts candidate tempos across multiple octaves ($T_1, T_2 = 0.5 T_1, T_3 = 2.0 T_1$).
- **Library:** `backend/beats/tempo_estimator.py` -> `estimate_tempo_hypotheses()`.
- **Output:** **TempoInfo** object containing candidate BPMs, primary BPM, autocorrelation curve, and confidence scores.

Stage 12: BPM Octave Disambiguation

- **Problem Solved:** Beat trackers frequently suffer octave errors (e.g. tracking an 86 BPM song at 172 BPM or 43 BPM).
- **Algorithm:** Evaluates harmonic rhythm transition rates and drum onset density to select the musically plausible base tempo before meter evaluation.

Stage 13: Ellis Dynamic Programming Beat Tracking

- **Input:** Onset envelope and selected base tempo BPM.
- **Algorithm:** Dan Ellis's dynamic programming beat tracker maximizing:

$$F(t) = O(t) + \max_{au} \left(F(t - au) - \alpha (\log(au / au_0))^2 \right)$$

where au_0 is the target beat period (60 / ext{BPM}) and α balances tempo consistency against onset alignment.

- **Library:** `librosa.beat.beat_track`.
- **File & Function:** `backend/beats/beat_tracker.py` -> `BeatTracker.track_beats()`.
- **Output:** Monotonically increasing array of beat timestamps $[t_0, t_1, \dots, t_{N-1}]$.

Stage 14: Downbeat Phase Tracking

- **Input:** Beat grid and onset/bass energy.
- **Algorithm:** Evaluates downbeat phase alignment $\phi \in \{0, \dots, M-1\}$ maximizing the correlation between candidate downbeat pulses and low-frequency spectral energy.
- **File & Function:** `backend/beats/downbeat_tracker.py` -> `track_downbeats()`.
- **Output:** Subsequence of beat timestamps classified as measure beginnings (downbeats).

Stage 15: Multi-Meter Detection (Core Metric Classifier)

- **Input:** `mono_22k` audio, beat grid, tempo hypotheses.
- **Algorithm:** Evaluates acoustic, harmonic, and rhythmic bar correlation across six distinct metric candidates: **2/4**, **3/4**, **4/4**, **6/8**, **7/8**, and **12/8**.
- **File & Function:** `backend/meter/meter_detector.py` -> `MeterDetector.detect_meter()`.
- **Output:** **MeterInfo** (numerator, denominator, display name, confidence, beat subdivisions).

Stage 16: 2/4 Time Signature Evaluation

- **Structure:** 2 quarter-note beats per bar.
- **Metric Indicator:** Strong alternating downbeat/upbeat alternating energy in 2-beat cycles without 4-beat structural hierarchies.

Stage 17: 3/4 Simple Triple Meter Evaluation

- **Structure:** 3 quarter-note beats per bar (Waltz / Triple time).
- **Metric Indicator:** Strong recurring energy every 3 beats ($O_t \gg O_{t+1}, O_{t+2}$).

- **Historical Milestone:** Corrected detection on *Bekhayali* (3/4 at 86.1 BPM), overriding the previous 4/4 misclassification (`docs/METER_TEMPO_FAILURE_REPORT.md`).

Stage 18: 4/4 Common Quadruple Meter Evaluation

- **Structure:** 4 quarter-note beats per bar (standard pop/rock metric structure with secondary accent on beat 3).

Stage 19: 6/8 Compound Duple Meter Evaluation

- **Structure:** 2 dotted-quarter macro beats divided into 2 times 3 eighth notes.

Stage 20: 7/8 Complex Additive Meter Evaluation

- **Structure:** 7 eighth-note pulses per measure.
- **Algorithm:** Multi-phase dynamic pulse evaluation testing asymmetric measure boundaries.

Stage 21: 12/8 Compound Quadruple Meter Evaluation

- **Structure:** 4 dotted-quarter macro beats divided into 4 times 3 eighth notes (standard blues/gospel slow triplet shuffle).

Stage 22: 7/8 Subgrouping Classification

- **Algorithm:** Evaluates acoustic onset accents across the 3 classic additive metric divisions:
 1. 2+2+3\$ (Daktilos pattern)
 2. 2+3+2\$ (Kalamatianos pattern)
 3. 3+2+2\$ (Makedonikos pattern)
- **Output:** Explicit subdivision string preserved in `MeterInfo.subdivisions` (e.g. `[2, 2, 3]`).

Stage 23: Bar Boundary Construction

- **Input:** Beat grid and verified meter numerator.
- **Algorithm:** Partitions the continuous beat grid into consecutive musical measures (`Bar` objects), computing precise `start_time` and `end_time` for each measure.

Stage 24: Krumhansl-Schmuckler Key Detection

- **Input:** Global chromagram averaged across the song.
- **Algorithm:** Correlates the 12-dimensional pitch class distribution against standard Krumhansl-Kessler cognitive key profiles for major and minor modes.
- **File & Function:** `backend/key/key_detector.py` -> `KeyDetector.detect_key()`.
- **Output:** `KeyInfo` (tonic note, mode, confidence).

Stage 25: Major / Minor Dual Scoring & Confidence

- **Algorithm:** Computes correlation score and margin between best major key and best minor key. Implements triad concentration weighting to prevent relative major/minor confusion (e.g. C Major vs. A Minor).

Stage 26: BTC Neural Network Architecture

- **Model:** Bidirectional Transformer for Chords (BTC).
- **Structure:** 8-layer Transformer encoder with multi-head self-attention operating over temporal frames.
- **File:** `backend/chord_recognition/btc_recognizer.py`, `models/btc/btc_model.py`.
- **Weights:** `models/btc/btc_model.onnx` (12.44 MB) and `btc_model_large_voca.pt` (11.66 MB).

Stage 27: BTC Preprocessing

- **Input:** `mono_22k` audio.
- **Algorithm:** Extracts logarithmic CQT features matching the exact training normalization parameters: mean subtraction and standard deviation scaling per frequency bin.

Stage 28: BTC Input Tensor Construction

- **Input Shape:** (B, T, F) where $B=1$ (batch size), T is time frames, and $F=144$ (CQT bins).
- **Execution:** Processed on CUDA GPU if available; otherwise falls back to multi-threaded CPU.

Stage 29: BTC 170-Class Chord Vocabulary

- **Classes:** 168 tonal chords + 1 "No Chord" (**N**) + 1 "Out of Vocabulary" (**X**).
- **Breakdown:** 12 root pitch classes $\times$ 14 chord qualities:
- `maj`, `min`, `dim`, `aug`, `maj7`, `min7`, `7`, `dim7`, `hmin7` (half-diminished), `minmaj7`, `maj6`, `min6`, `sus2`, `sus4`.

- **File:** `backend/chord_recognition/vocabulary.py`.

Stage 30: BTC Forward Inference & Posterior Probabilities

- **Algorithm:** Softmax activation over Transformer logits yields a $(T, 170)$ posterior probability matrix representing the probability distribution over all 170 chords for every 23.2 ms frame.
- **Dual Inference:** Executed twice per song:
 1. Once on the Original Mix (**mono_22k**).
 2. Once on the Vocal-Free Accompaniment Stem (**other.wav**).

Stage 31: Sub-Bass Fundamental Extraction

- **Input:** Isolated **bass.wav** stem.
- **Algorithm:** Sharp low-pass Chebyshev filter ($f_c = 250$ Hz) isolates the physical bass register. Employs parabolic-interpolated autocorrelation to determine the sounding fundamental frequency $f_0(t)$ and maps it to the nearest MIDI note:

$$p = 69 + 12 \log_2(f_0 / 440)$$

- **File & Function:** `backend/bass/inversion_detector.py` -> `BassStemAnalyzer.analyze_bass_track()`.

Stage 32: Inversion Detection

- **Input:** Detected harmony chord root R , third 3^{rd} , fifth 5^{th} , and physical bass note $B(t)$.
- **Logic:**
 - If $B = R \pmod{12}$: Root position (standard chord).
 - If $B = 3^{\text{rd}} \pmod{12}$: First inversion (e.g. C/E).
 - If $B = 5^{\text{th}} \pmod{12}$: Second inversion (e.g. C/G).
 - If $B = 7^{\text{th}} \pmod{12}$: Third inversion (e.g. C7/Bb).

Stage 33: Slash Chord Formatting

- **Algorithm:** If sounding bass note B differs from chord root R with confidence > 0.60 , formats chord display as **Root/Bass** (e.g. **D/F#**, **Am/G**).

Stage 34: Passing-Bass Filtering

- **Algorithm:** Transient bass notes lasting less than 1.5 beat periods are classified as walking/passing bass lines and do not alter the underlying harmonic chord label.

Stage 35: Beat-Synchronous Multi-Source Ensemble Fusion

- **Algorithm:** Combines predictions from Original Mix, Accompaniment Stem, and Bass Stem:

$$P_{\text{fused}}(c, b) = w_{\text{mix}} P_{\text{mix}}(c, b) + w_{\text{other}} P_{\text{other}}(c, b)$$

where probabilities are pooled across beat windows $[t_i, t_{i+1})$.

- **File & Function:** `backend/chord_recognition/btc_recognizer.py` -> `EnsembleChordAnalyzer.fuse_beat_probabilities()`.

Stage 36: Temporal Smoothing

- **Algorithm:** Viterbi dynamic programming or median filter over beat predictions eliminates rapid spurious chord switching ("chatter").

Stage 37: Simplicity Regularization

- **Algorithm:** If a complex chord (e.g. **Cmaj7**) has marginal probability advantage over its diatonic triad parent (**C**), the simpler triad is selected unless the 7th extension is acoustically prominent ($P_{\text{ext}} > 0.40$).

Stage 38: Bar Consolidation & Chord Snapping

- **Input:** Beat-synchronous chord sequence and bar boundaries.
- **Algorithm:** Consecutive identical chords within the same measure are consolidated. Chords spanning less than 0.5 beats are merged into dominant measure harmony, eliminating edge-overlap blips.

Stage 39: Recurrence Matrix Structural Section Discovery

- **Input:** 12-bin chromagram and MFCC timbre features across measures.
- **Algorithm:** Computes cross-similarity recurrence matrix $R(i, j) = \cos(\mathbf{v}_i, \mathbf{v}_j)$ between bars. Diagonal path clustering segments the song into structural blocks with neutral labels (**SECTION A**, **SECTION B**, **INTRO**, **OUTRO**).
- **File & Function:** `backend/sections/section_detector.py` -> `SectionDetector.detect_sections()`.

Stage 40: Final SongAnalysis JSON Assembly & Schema Validation

- **Input:** Metadata, tempo info, meter info, beat grid, bars, chords, sections, key info.
- **Algorithm:** Instantiates Pydantic **SongAnalysis** model and validates against **shared/music_schema/song_analysis.schema.json**. Returns complete JSON contract to caller.

SECTION 04

Chapter 4: Android Mobile Architecture & Flutter Client

Android Mobile Architecture Blueprint

1. Executive Architecture Overview

The Android application for Song Chord Analyzer is built with **Flutter / Dart** targeting Android 8.0+ (API level 26+). It operates as a high-performance, reactive presentation client backed by the authoritative Windows Python MIR server.

```
flowchart TD
    subgraph MobileApp["Flutter Android Application (com.songchordanalyzer.app)"]
        subgraph UI["UI Screens"]
            HomeScreen["Home Screen (Audio File Picker)"]
            ProgressScreen["Analysis Progress Screen (Real-time Polling)"]
            ChordScreen["Chord Sheet Screen (Responsive 2-Col Grid)"]
            HistoryScreen["History Screen (Offline Saved Songs)"]
            SettingsScreen["Settings Screen (Connection Modes)"]
        end

        subgraph ServicesTier["Client Services Tier"]
            ServerConfig["ServerConfigService (Local / Remote / Auto)"]
            HttpEngine["DevHttpAnalysisEngine (Upload & Polling)"]
            PlaybackSvc["AudioPlayerService (audioplayers)"]
            TransposeSvc["TransposeService (Real-time Pitch Shift)"]
            HistorySvc["HistoryService (sqflite Local DB)"]
            ExportSvc["ExportService (Single-Page Musician PDF)"]
        end

        subgraph StorageTier["On-Device Local Storage"]
            SQLiteMobile["SQLite Database (songs.db)"]
            AudioCache["Local Audio Cache (app_flutter/audio/)"]
            Prefs["SharedPreferences (Config & Modes)"]
        end
    end

    subgraph HostServer["Host Compute Server (Laptop / PC)"]
        FastAPIEndpoint["FastAPI Server (Port 8000)"]
    end

    HomeScreen -->|Selects Audio| HttpEngine
    SettingsScreen -->|Configures Mode| ServerConfig
    ServerConfig -->|Resolves Base URL| HttpEngine
    HttpEngine -->|Multipart Upload & Poll| FastAPIEndpoint
    FastAPIEndpoint -->|SongAnalysis JSON & Audio| HttpEngine
    HttpEngine -->|Parses Analysis| ChordScreen
    HttpEngine -->|Saves Offline Copy| HistorySvc

    HistorySvc --> SQLiteMobile
    HistorySvc --> AudioCache
    ServerConfig --> Prefs

    ChordScreen --> PlaybackSvc
    ChordScreen --> TransposeSvc
    ChordScreen --> ExportSvc
```

2. Screen-by-Screen Technical Reference

2.1 Home Screen (HomeScreen)

- **File:** `mobile/flutter_app/lib/screens/home_screen.dart`.
- **Purpose:** Primary application dashboard. Allows users to select an audio file, initiate analysis, view the active connection mode status pill, or jump to recent songs.
- **Widgets Used:** `ConnectionStatusBanner`, `ElevatedButton`, `RecentSongsList`, `FilePicker`.
- **Interactions:** Uses `file_picker` package to browse local device storage for `.mp3`, `.wav`, `.m4a`, and `.flac`. Upon selection, navigates to `AnalysisProgressScreen`.

2.2 Analysis Progress Screen (AnalysisProgressScreen)

- **File:** `mobile/flutter_app/lib/screens/analysis_progress_screen.dart`.

- **Purpose:** Visualizes real-time progress while the host server executes the 12-stage MIR pipeline.
- **State Management:** Polls **GET /api/analyze/{analysis_id}** every 500 ms using a repeating **Timer**.
- **UI Elements:** Circular progress indicator, percentage label (**0%** to **100%**), and localized stage description ("**Separating stems with Demucs...**", "**Estimating beat grid...**", etc.).
- **Completion Flow:** Upon receiving status **COMPLETED**, deserializes **SongAnalysis**, caches it in local SQLite via **HistoryService**, and navigates to **ChordSheetScreen**.

2.3 Chord Sheet Screen (ChordSheetScreen)

- **File:** `mobile/flutter_app/lib/screens/chord_sheet_screen.dart`.
- **Purpose:** Core musician interface displaying the interactive chord sheet.
- **Responsive Layout:** Automatically detects screen width. On mobile portrait devices (≤ 600 ext{ px}), renders a high-density **2-column grid**; on tablets or landscape (> 600 ext{ px}), expands to 4 columns.
- **Active Tracking:** Synchronizes with **AudioPlayerService** audio position stream. As music plays, highlights the active measure and active chord chip with an accent border and auto-scrolls the viewport to keep the playing bar in view.
- **Controls:** Floating audio player bar, transpose bottom sheet (**-11** to **+11** semitones), and export menu (PDF / TXT).

2.4 History Screen (HistoryScreen)

- **File:** `mobile/flutter_app/lib/screens/history_screen.dart`.
- **Purpose:** Displays all previously analyzed songs stored in on-device SQLite.
- **Offline Capability:** Operates **100% offline**. Users can open songs, play cached audio, transpose chords, and export PDFs without any connection to the laptop server.

2.5 Settings Screen (SettingsScreen)

- **File:** `mobile/flutter_app/lib/screens/settings_screen.dart`.
- **Purpose:** Controls network connection configuration across Local LAN, Remote Internet, and Auto modes. Includes real-time connection diagnostic probes measuring ping latency in milliseconds.

3. Custom Widgets & Presentation Components

- **BarView** (`mobile/flutter_app/lib/widgets/bar_view.dart`): Renders an individual musical bar. Displays bar number (e.g. **Bar 12**), time signature tag (**4/4**, **7/8**), and chord chips. Highlights with an orange border when the playback timeline enters the bar's [**start_time**, **end_time**] window.
- **ChordTile** (`mobile/flutter_app/lib/widgets/chord_tile.dart`): Displays an individual chord label (e.g. **Dm**, **C/E**). Renders a compact chip with distinct styling for the active playing chord.
- **AudioPlaybackBar** (`mobile/flutter_app/lib/widgets/audio_playback_bar.dart`): Fixed bottom player bar providing Play/Pause, Seek Slider, Elapsed/Total time counters, and playback speed controls (**0.75x**, **1.0x**, **1.25x**).
- **ChordEditorModal** (`mobile/flutter_app/lib/widgets/chord_editor_modal.dart`): Bottom sheet modal allowing musicians to manually override or correct individual chords.

4. Client Services Tier

4.1 ServerConfigService

- **File:** `mobile/flutter_app/lib/services/server_config_service.dart`.
- **Role:** Singleton ChangeNotifier managing server URLs and modes. Persists settings in **SharedPreferences**.
- **Connection Modes:**
- **ConnectionMode.local:** Points to **http://192.168.x.x:8000**.
- **ConnectionMode.remote:** Points to **https://*.trycloudflare.com**.
- **ConnectionMode.auto:** Probes Local LAN first with 1.5s timeout; falls back to Remote if unreachable.
- **Diagnostics:** **testConnection(url, apiKey)** sends a **GET /api/health** request and returns status code, response payload, and round-trip ping time.

4.2 DevHttpAnalysisEngine

- **File:** `mobile/flutter_app/lib/services/dev_http_analysis_engine.dart`.
- **Role:** Implements the **AnalysisEngine** interface. Handles multipart file uploads, job queue polling, and deserialization of **SongAnalysis** JSON.

4.3 AudioPlayerService

- **File:** `mobile/flutter_app/lib/services/audio_player_service.dart`.
- **Role:** Wraps the `audioplayers` package. Emits continuous streams for `positionStream`, `durationStream`, and `playerStateStream`. Handles local file URIs as well as remote audio streaming URLs (`/api/analysis/{id}/audio`).

4.4 TransposeService

- **File:** `mobile/flutter_app/lib/services/transpose_service.dart`.
- **Role:** Pure client-side transposition engine. Takes a `SongAnalysis` object and semitone shift offset `$k \in [-11, +11]`, recalculates all chord roots and slash bass notes using chromatic modular arithmetic, and preserves enharmonic spelling based on the transposed key tonic.

4.5 HistoryService

- **File:** `mobile/flutter_app/lib/services/history_service.dart`.
- **Role:** Local persistence layer using `sqflite`. Stores complete serialized `SongAnalysis` JSON strings alongside metadata in `songs.db`. Caches downloaded audio files in the device application documents directory.

4.6 ExportService

- **File:** `mobile/flutter_app/lib/services/export_service.dart`.
- **Role:** Vector PDF and plain text generator. Uses the Dart `pdf` package to construct the musician-grade lead sheet and executes the measurement-based Fit-to-Page algorithm to guarantee a single-page document.

5. Audit of Retired On-Device Kotlin Engine

In early development (Phase 2), on-device analysis was attempted in Kotlin (`mobile/flutter_app/android/app/src/main/kotlin/com/songchordanalyzer/app/`). This engine has been formally retired and replaced by the server-backed architecture.

Why On-Device Analysis Was Retired:

1. **Demucs RAM Limitation:** Demucs stem separation requires > 2 GB RAM and heavy PyTorch tensor math. Mobile devices experienced frequent out-of-memory crashes (`java.lang.OutOfMemoryError`).
2. **CQT Filterbank Discrepancy:** The native Kotlin CQT implementation (`CqtExtractor.kt`) used an approximation that diverged from Librosa's CQT filterbank, resulting in degraded chord accuracy.
3. **Key Detection Tritone Bug:** In `KeyDetector.kt`, bin indexing `chroma[b % 12] += mag` on a 24-bin/octave CQT mapped bin 0 (C) and bin 12 (F#) to the same pitch class, completely scrambling key classification.
4. **Meter / Tempo Divergence:** The mobile Ellis beat tracker diverged from the Windows reference implementation on complex triple and additive meters.

Current Status: Kotlin source files are retained in the repository solely for historical documentation and potential future lightweight on-device experiments. Production analysis is 100% routed through the authoritative server.

SECTION 05

Chapter 5: Windows Desktop Architecture & Electron Shell

Windows Desktop Architecture Blueprint

1. Executive Desktop Architecture

The Windows desktop application provides a standalone, zero-configuration executable environment. It packages an **Electron desktop shell**, a **React/Vite web user interface**, and the **authoritative Python MIR backend** into a unified distribution.

```

flowchart TD
    subgraph WindowsDesktopApp["Windows Desktop Application (SongChordAnalyzer.exe)"]
        subgraph ElectronHost["Electron Host Process (electron/main.cjs)"]
            PortFinder["Dynamic Port Discovery (findFreePort: 8000+)"]
            PythonLauncher["Python Process Spawner (run_app.py --no-browser)"]
            HealthProber["HTTP Health Poller (GET /api/health)"]
            Splash["Native Splash Window (Loading Status)"]
            MainWindow["Main Browser Window (React UI)"]
        end

        subgraph EmbeddedRuntime["Embedded Runtime Environment"]
            PythonVenv["Isolated Python Runtime (StemKit / AppData venv)"]
            FFmpegBinary["Bundled FFmpeg (resources/ffmpeg/ffmpeg.exe)"]
            ModelWeights["Pretrained Weights (models/btc/)"]
        end

        subgraph WebFrontend["React / Vite Frontend (frontend/dist/)"]
            UIState["React Component State & Context"]
            Waveform["Waveform Audio Player"]
            DesktopSheet["Desktop Chord Sheet & Editor"]
        end
    end

    ElectronHost --> PortFinder
    PortFinder --> PythonLauncher
    PythonLauncher --> EmbeddedRuntime
    PythonLauncher --> HealthProber
    HealthProber -->|200 OK| MainWindow
    MainWindow --> WebFrontend
    WebFrontend -->|REST API over localhost:8000| PythonLauncher

```

2. Electron Lifecycle & Process Management (electron/main.cjs)

2.1 Dynamic Port Discovery (findFreePort)

- To prevent port collision with other local services (such as development servers or Docker), **findFreePort(8000)** creates a temporary TCP server on port 8000. If port 8000 is occupied, it requests an ephemeral free port from the OS kernel.

2.2 Python Executable Resolution (resolvePythonExecutable)

The launcher locates the Python runtime by querying the following priority cascade:

- Environment variable **SONG_CHORD_ANALYZER_PYTHON**.
- Packaged Electron resources directory: **process.resourcesPath/python/python.exe**.
- Local application resources directory: **resources/python/python.exe**.
- Windows AppData isolated runtime: **%LOCALAPPDATA%/SongChordAnalyzer/venv/Scripts/python.exe** or **%APPDATA%/StemKit/venv/Scripts/python.exe**.
- System PATH: **python.exe**.

2.3 Process Spawning & Health Polling

- Electron spawns the backend process:

```
python run_app.py --port <port> --no-browser
```

- While Python initializes PyTorch, CUDA, and FastAPI, Electron displays a clean **native splash window**.
- Electron polls **http://127.0.0.1:<port>/api/health** every 250 ms.

- Once the backend reports "**status**": "**healthy**", the splash window closes and the main browser window loads **frontend/dist/index.html?port=<port>**.

2.4 Graceful Process Termination

- When the user closes the desktop window, Electron intercepts **before-quit** and **window-all-closed**.
- On Windows, it issues a **taskkill /pid <pid> /f /t** command to ensure the Python process and any FFmpeg child processes terminate cleanly, preventing orphaned zombie processes.

3. Web Frontend Architecture (frontend/)

- **Tooling**: Built with **Vite 5.4**, **React 18**, and **TypeScript**.
- **State Management**: Reactive component state managing song upload, playback timeline, active chord indices, and transposition.
- **Waveform Synchronization**: Visual audio scrubber tracking current playback time and synchronizing with chord bar measure boundaries.
- **Styling**: Tailwind CSS with dark-mode optimized color tokens for high readability in stage and studio environments.

4. Windows Reference Engine (backend/pipeline.py)

The Windows backend implementation is the **authoritative gold-standard reference** for the entire project:

- Uses PyTorch with native NVIDIA CUDA acceleration on the host GPU.
- Directly invokes Demucs v4 hybrid transformer for 4-stem source separation.
- Runs the complete BTC Transformer model with 170 chord classes.
- Tracks sub-bass register fundamentals ($\$ < 250 \text{ ext} \{ \text{Hz} \} \$$) on the isolated bass stem to resolve slash chord inversions (e.g. **C/E**, **G/B**).
- **Parity Validation**: Validated with 100% mathematical parity against the FastAPI server on *Bekhayali* (**WINDOWS_SERVER_PARITY_REPORT.md**).

5. Packaging & Distribution Configurations

- **Tool**: **electron-builder** configured via **electron-builder.yml**.
- **Application Icon**: **build/app.ico** (256x256 multi-resolution icon).
- **Distribution Outputs**:
 1. **NSIS Setup Installer**: **dist_electron/SongChordAnalyzer-Setup.exe** (142.78 MB). Provides desktop shortcut, start menu entry, and clean uninstaller.
 2. **Portable Executable**: **dist_electron/SongChordAnalyzer.exe** (142.57 MB). Standalone portable package requiring no administrative installation rights.

SECTION 06

Chapter 6: FastAPI Server & REST API Reference

FastAPI Server & REST API Reference

The Song Chord Analyzer backend exposes a REST API powered by **FastAPI 1.0.0** running on **http://0.0.0.0:8000**.

1. Authentication & Security Middleware

- **Optional API Key Authentication:** If the host sets the environment variable **API_AUTH_KEY** or **SONG_CHORD_ANALYZER_API_KEY**, the server enforces authentication across all data and analysis routes.
- **Header:** **X-API-Key:** <secret_key>
- **Exempt Paths:** **/health**, **/api/health**, **/docs**, **/openapi.json**, and **/assets/*** are exempt to allow diagnostic connection probes and static asset rendering.
- **Rejection:** Unauthorized requests receive **HTTP 401 Unauthorized** with JSON payload:

```
{"detail": "Unauthorized: Invalid or missing X-API-Key header"}
```

2. Complete Endpoint Specification

2.1 System Health & Hardware Diagnostics

- **Path:** **GET /api/health** (also aliased at **GET /health**)
- **Authentication:** None (Public diagnostic probe)
- **Response Schema (HealthCheckResponse):**

```
{
  "status": "healthy",
  "api_version": "1.0.0",
  "schema_version": "1.0.0",
  "analysis_engine": "WindowsAnalysisEngine (Authoritative MIR Pipeline)",
  "analysis_engine_available": true,
  "auth_required": false,
  "models_available": {
    "btc": true,
    "demucs": true
  },
  "device": "NVIDIA GeForce RTX 3050 6GB Laptop GPU",
  "cuda_available": true,
  "cpu_count": 16,
  "ram_total_gb": 15.7,
  "vram_free_mb": 5161.0,
  "vram_total_mb": 6143.5,
  "ffmpeg_path": "C:\\...\\resources\\ffmpeg\\ffmpeg.exe",
  "python_executable": "C:\\...\\python.exe"
}
```

2.2 Submit Audio for Analysis

- **Path:** **POST /api/analyze**
- **Content-Type:** **multipart/form-data**
- **Request Parameters:**
- **file:** Audio file binary (MP3, WAV, FLAC, M4A, AAC). Maximum size: **100 MB**.
- **title (optional):** String title override.
- **Admission Control:** If active in-flight analysis jobs $\geq$ 2\$, returns **HTTP 503 Service Unavailable**:

```
{"detail": "Analysis queue is full (2 jobs in flight). Please wait for current analyses to..."}
```

- **Response (202 Accepted):**

```
{
  "analysis_id": "b617036d",
  "status": "queued",
  "progress": 0,
  "current_stage": "QUEUED",
  "message": "Audio received successfully, queued for analysis...",
  "created_at": "2026-10-08T08:17:02.123456Z"
}
```

2.3 Poll Analysis Job Status

- **Path:** GET /api/analyze/{analysis_id}
- **Path Parameter:** analysis_id (string UUID8).
- **Response (AnalysisStatusResponse):**

```
{
  "analysis_id": "b617036d",
  "status": "separating",
  "progress": 25,
  "current_stage": "SEPARATING",
  "message": "Separating stems with Demucs (Bass + Accompaniment)...",
  "created_at": "2026-10-08T08:17:02.123456Z",
  "started_at": "2026-10-08T08:17:02.456789Z",
  "completed_at": null,
  "error": null
}
```

- **Possible status values:** queued, preprocessing, separating, analyzing_beats, analyzing_chords, analyzing_inversion, analyzing_key, aligningBars, detecting_sections, completed, failed.

2.4 Retrieve Completed SongAnalysis

- **Path:** GET /api/analysis/{analysis_id}
- **Response:** Complete SongAnalysis JSON contract (see [DATA_SCHEMA.md](#)).
- **Errors:** HTTP 404 Not Found if analysis ID does not exist or has not completed.

2.5 Audio Playback Streaming

- **Path:** GET /api/analysis/{analysis_id}/audio
- **Headers Supported:** Range: bytes=start-end (supports standard HTTP 206 Partial Content for audio seeking).
- **Content-Type:** audio/mpeg or audio/wav.
- **Side Effect:** Streams the ingested library audio file associated with analysis_id.

2.6 Server-Side Transposition

- **Path:** POST /api/analysis/{analysis_id}/transpose
- **Request Body (TransposeRequest):**

```
{
  "semitones": 2
}
```

- **Response:** Transposed SongAnalysis object.

2.7 Song History & Library

- **GET /api/history:** Returns paginated list of all analyzed songs in SQLite.
- **GET /api/history/recent:** Returns recent analyzed songs (limit: 10).
- **POST /api/history/{song_id}/favorite:** Toggles favorite status.
- **DELETE /api/history/{song_id}:** Deletes song from database and managed audio library.

2.8 Server-Side Document Export

- **GET /api/analysis/{analysis_id}/export/pdf:** Returns binary PDF lead sheet formatted with ReportLab.
- **GET /api/analysis/{analysis_id}/export/txt:** Returns plain text musician chart.
- **GET /api/analysis/{analysis_id}/export/json:** Returns raw SongAnalysis JSON.

SECTION 07

Chapter 7: Data Schema Contract (SongAnalysis v1.0.0)

Data Schema Contract (SongAnalysis v1.0.0)

The **SongAnalysis** JSON schema is the **central shared contract** defining musical data exchange across the Python MIR backend, FastAPI server, Flutter Android application, and Electron desktop shell.

Formal Specification: `shared/music_schema/song_analysis.schema.json`

JSON Schema Dialect: JSON Schema Draft 2020-12

Contract Version: `1.0.0`

1. Root Object Hierarchy

```
SongAnalysis
  schema_version: "1.0.0"
  id: string (UUID8)
  title: string
  audio_url: string (optional)
  metadata: AudioMetadata
  pipeline_metadata: PipelineMetadata
  key: KeyInfo
  tempo: TempoInfo
  meter: MeterInfo
  beat_grid: BeatGrid
  sections: MusicalSection[]
  chords: ChordPrediction[]
  bars: Bar[]
  raw_predictions: RawPrediction[] (optional)
  debug_view: DebugBarEntry[] (optional)
```

2. Field-by-Field Technical Reference

2.1 Metadata Objects

metadata (AudioMetadata)

- filename** (*string, required*): Original filename (e.g. `"BekhayaLi.mp3"`).
- duration** (*number, required, ≥ 0*): Total audio duration in seconds.
- sample_rate** (*integer, required*): Audio sample rate in Hz (e.g. `44100` or `22050`).
- channels** (*integer, required*): Channel count (`1` = mono, `2` = stereo).
- format** (*string, required*): Container format (`"mp3"`, `"wav"`, etc.).
- file_size_bytes** (*integer, optional*): Audio file size on disk.
- file_hash** (*string, optional*): SHA-256 audio content digest.

pipeline_metadata (PipelineMetadata)

- app_version** (*string*): Application version (e.g. `"1.0.0"`).
- model_name** (*string*): Chord recognition model identifier (`"BTC-Transformer"`).
- model_version** (*string*): Model weights version (`"large_voca_170"`).
- separation_model** (*string*): Stem separation architecture (`"htdemucs_v4"`).
- device_used** (*string*): Hardware execution target (`"cuda:0"` or `"cpu"`).
- timestamp** (*string*): ISO 8601 UTC analysis timestamp.

2.2 Harmonic & Metric Descriptors

key (KeyInfo)

- tonic** (*string, required*): Root note letter (e.g. `"Bb"`, `"C"`, `"F#"`).
- mode** (*string, required*): `"major"` or `"minor"`.
- display** (*string, required*): Formatted key string (e.g. `"Bb Minor"`).

- **confidence** (*number, required, \$0.0 \le c \le 1.0\$*): K-S profile correlation confidence score.

tempo (TempoInfo)

- **bpm** (*number, required, \$30.0 \le ext{bpm} \le 300.0\$*): Master tempo in beats per minute (e.g. **86.1**).
- **confidence** (*number, required*): Autocorrelation peak confidence score.
- **candidate_bpms** (*number[], optional*): Multi-hypothesis tempo candidates (e.g. [**43.05**, **86.1**, **172.2**]).
- **beat_period** (*number, optional*): Average duration between beats ($\$60.0 / ext{bpm}\$$).

meter (MeterInfo)

- **numerator** (*integer, required, \$\ge 1\$*): Beats per measure (**2**, **3**, **4**, **6**, **7**, **12**).
- **denominator** (*integer, required, \$\ge 1\$*): Note value receiving one beat (**4** or **8**).
- **display** (*string, required*): Standard time signature string (e.g. "**4/4**", "**3/4**", "**7/8**").
- **confidence** (*number, required*): Metric classification confidence score.
- **subdivisions** (*integer[], optional*): Additive meter subdivisions (e.g. [**2**, **2**, **3**] for 7/8).

2.3 Rhythmic Grids & Structure

beat_grid (BeatGrid)

- **bpm** (*number, required*): Calibrated tempo.
- **beats** (*number[], required*): Array of floating-point timestamps in seconds for every detected musical beat.
- **downbeats** (*number[], required*): Array of timestamps marking the first beat of each musical bar.
- **confidence** (*number, optional*): Overall beat tracking reliability metric.

sections (MusicalSection[])

- Array of structural sections detected via recurrence matrix clustering:
- **name** (*string*): Neutral structural label ("**INTRO**", "**SECTION A**", "**SECTION B**", "**OUTRO**").
- **start_time** (*number*): Start time in seconds.
- **end_time** (*number*): End time in seconds.
- **bar_start** (*integer*): 0-indexed starting bar number.
- **bar_end** (*integer*): 0-indexed ending bar number.

2.4 Chords & Bar Measures

chords (ChordPrediction[])

Continuous array of recognized chords aligned to beat intervals:

- **root** (*string, required*): Chord root note (e.g. "**D**", "**Bb**").
- **quality** (*string, required*): Harmonic quality ("**maj**", "**min**", "**7**", "**sus4**", etc.).
- **bass** (*string, optional*): Physical sounding bass note for slash chords (e.g. "**F#**" in **D/F#**).
- **display** (*string, required*): Formatted chord label (e.g. "**Dm**", "**Asus4**", "**C/E**").
- **start_time** (*number, required*): Chord start timestamp in seconds.
- **end_time** (*number, required*): Chord end timestamp in seconds.
- **confidence** (*number, required*): Posterior probability score ($\$0.0 \le c \le 1.0\$$).

bars (Bar[])

Structured musical measures for chord sheet and PDF formatting:

- **bar_number** (*integer, required*): 0-indexed measure index (**0**, **1**, **2**, ...).
- **start_time** (*number, required*): Exact measure start timestamp.
- **end_time** (*number, required*): Exact measure end timestamp.
- **chords** (*ChordPrediction[], required*): Chords sounding within this specific measure.
- **time_signature** (*string, optional*): Time signature override if metric modulation occurs.

SECTION 08

Chapter 8: Dual-Mode Networking & Connection Architecture

Dual-Mode Networking & Connection Architecture

Song Chord Analyzer features a **dual-mode networking architecture** allowing Android devices to communicate with the host laptop server seamlessly in any environment.

```

flowchart TD
    subgraph MobileDevice["Android Phone (Flutter App)"]
        ClientService["ServerConfigService"]
    end

    subgraph Modes["Connection Modes"]
        LocalMode["1. Local LAN Mode (Home Wi-Fi / Hotspot)"]
        RemoteMode["2. Remote Internet Mode (Cellular 4G/5G)"]
        AutoMode["3. Auto Failover Mode"]
    end

    subgraph HostLaptop["Host Laptop / PC (FastAPI Server)"]
        LocalPort["http://192.168.x.x:8000 (LAN Listener)"]
        TunnelClient["tools/cloudflared.exe (Encrypted Proxy)"]
        FastAPICore["FastAPI REST Engine (127.0.0.1:8000)"]
    end

    ClientService -->|User Selects| LocalMode
    ClientService -->|User Selects| RemoteMode
    ClientService -->|User Selects| AutoMode

    LocalMode -->|Direct HTTP Traffic| LocalPort
    RemoteMode -->|Public HTTPS (*.trycloudflare.com)| TunnelClient
    AutoMode -->|1.5s Probe: Success| LocalPort
    AutoMode -->|1.5s Probe: Timeout| TunnelClient

    LocalPort --> FastAPICore
    TunnelClient --> FastAPICore

```

1. Connection Mode Technical Details

1.1 Mode 1: Local Network Mode (LAN / Wi-Fi)

- **Transport:** Standard HTTP over local IPv4 private addresses (**http://192.168.x.x:8000**).
- **Use Case:** Home, rehearsal space, or studio where phone and laptop share the same Wi-Fi router or phone mobile hotspot.
- **Internet Requirement: Zero Internet Required.** Operates 100% offline as long as the local network links phone and PC.
- **Latency:** Extremely low (5 ms - 25 ms).
- **Upload Speed:** Maximum local Wi-Fi throughput (a 10 MB audio file transfers in < 1 second).
- **Windows Firewall Configuration:**

If connection is blocked, allow inbound TCP traffic on port 8000:

```
netsh advfirewall firewall add rule name="Song Chord Analyzer Backend" dir=in action=allow...
```

1.2 Mode 2: Remote Internet Mode (Cloudflare Tunnel)

- **Transport:** Encrypted TLS (HTTPS) over Cloudflare's global edge network (**https://*.trycloudflare.com**).
- **Use Case:** Away from home, traveling, or using mobile cellular data (4G/5G).
- **Tool:** Bundled static binary **tools/cloudflared.exe**.
- **Execution Script:** **scripts/start_tunnel.bat**.

```
cloudflared.exe tunnel --url http://127.0.0.1:8000
```

- **How It Works:** **cloudflared** creates an outbound encrypted QUIC/HTTP2 tunnel from the laptop to Cloudflare's nearest edge node and assigns an ephemeral public URL. Requests to this URL are routed over the tunnel directly to the laptop's local port 8000 without requiring port forwarding, static IPs, or router changes.
- **Security & Privacy:** No router ports are opened to the public internet. All traffic is encrypted end-to-end. Optional **X-API-Key** authentication prevents unauthorized compute usage.

- **Crucial Fact: The laptop still performs 100% of the computation.** Cloudflare acts purely as an encrypted transport layer. If the laptop is asleep or disconnected, remote requests fail.

1.3 Mode 3: Auto Connection Mode

- **Mechanism:**
 1. The app issues an asynchronous probe request to **GET /api/health** at the configured Local URL with a **1.5-second timeout**.
 2. If the local server responds with HTTP 200, the app binds to the Local LAN connection (prioritizing high speed and zero internet dependency).
 3. If the local probe times out or fails (indicating the user has left the home network), the app seamlessly switches to the Remote Tunnel URL.
 4. The active connection mode is displayed as a visual badge on the home screen banner.
-

2. Real-Time Connection Diagnostics

In the mobile **Settings Screen** (`mobile/flutter_app/lib/screens/settings_screen.dart`):

- Tapping "**Test Connection**" executes a round-trip diagnostic against **/api/health**.
- The app measures exact network latency in milliseconds.
- Displays hardware telemetry returned by the server:
- Host GPU model (e.g. **NVIDIA GeForce RTX 3050 Laptop GPU**).
- VRAM available (**5,161 MB / 6,143 MB**).
- Device analysis engine status (**WindowsAnalysisEngine**).

SECTION 09

Chapter 9: Build, Packaging & Release Engineering

Build, Packaging & Release Engineering

This document details the exact commands, toolchains, and packaging pipelines used to build production artifacts for Windows desktop, Android mobile, and containerized server environments.

1. Android Mobile APK Build

1.1 Prerequisites

- Flutter SDK 3.24+ (**C:\Users\jerin\flutter\bin\flutter.bat**).
- Android SDK (API 34 / Java 17).
- Pre-trained quantized ONNX model in assets: **mobile/flutter_app/assets/models/btc_model_170voca.onnx**.

1.2 Build Commands

From repository root:

```
cd mobile/flutter_app

# 1. Clean build directory & fetch packages
flutter clean
flutter pub get

# 2. Run static analysis & test suite
flutter analyze
flutter test

# 3. Build optimized release APK
flutter build apk --release

# 4. Build release APK with hardcoded default remote URL (Optional)
flutter build apk --release --dart-define=API_BASE_URL=https://your-tunnel.trycloudflare.com
```

1.3 Artifact Output

- Raw Gradle Output:** **mobile/flutter_app/build/app/outputs/flutter-apk/app-release.apk**
- Root Release Copy:** **SongChordAnalyzer.apk** (123.92 MB).
- Contents:** Stripped Flutter AOT runtime, JNI shared libraries (**libapp.so**, **libflutter.so**, **libonnxruntime.so**), bundled assets, fonts, and icons.

2. Windows Desktop Executable Build

2.1 Prerequisites

- Node.js 18+ and npm (**package.json**).
- Python 3.11 with virtual environment containing PyTorch, Librosa, and Demucs.
- Application icon at **build/app.ico**.

2.2 Build Commands

From repository root:

```
# 1. Install desktop & frontend dependencies
npm install
npm run frontend:install

# 2. Compile React frontend into static bundle (frontend/dist/)
npm run build:frontend

# 3. Package Windows desktop application via electron-builder
npm run dist
```

2.3 Packaging Specification (electron-builder.yml)

- **Target Formats:**
 1. **nsis** (x64 Windows Setup Installer) -> **dist_electron/SongChordAnalyzer-Setup.exe** (142.78 MB).
 2. **portable** (x64 Standalone Executable) -> **dist_electron/SongChordAnalyzer.exe** (142.57 MB).
- **Bundled Resources:** Automatically packages **resources/ffmpeg/ffmpeg.exe**, **models/btc/**, and unpacked backend source files.
- **Signing:** Uses **electron/sign-noop.cjs** for unsigned local development builds.

3. Backend Docker Container Build

3.1 Dockerfile Overview (Dockerfile)

- Base image: **python:3.11-slim** with system FFmpeg, libsndfile1, and git installed.
- Exposes port **8000**.
- Health check: Probes **http://localhost:8000/api/health** every 30 seconds.

3.2 Build & Run Commands

```
# 1. Build Docker image
docker build -t song-chord-analyzer-backend .

# 2. Run container with GPU acceleration
docker run -d --gpus all -p 8000:8000 -v song_data:/app/storage --name song-analyzer song-ch...

# 3. Run container in CPU mode
docker run -d -p 8000:8000 -v song_data:/app/storage --name song-analyzer song-chord-analyze...
```

SECTION 10

Chapter 10: Testing, Verification & Parity Validation

Testing, Verification & Parity Validation

This document catalogs the complete automated testing architecture, verification evidence, and parity proofs across Song Chord Analyzer.

1. Complete Test Suite Matrix

Test Suite	Framework / Tool	Test File	Test Count	Current Status	Execution Time
Python Backend Tests	unittest / pytest	tests/test_*.py	44 tests	100% PASS (OK)	~65s (CPU/CUDA)
Flutter Mobile Tests	flutter test	mobile/flutter_app/test/	16 tests	100% PASS (OK)	~5s
Flutter Code Analysis	flutter analyze	mobile/flutter_app/	0 issues	100% PASS	~10s
Windows <-> FastAPI Parity	Standalone runner	tests/server_parity/test_windows_server_parity.py	21 checks	100% PARITY	~36s
Golden Meter Regression	unittest	tests/test_golden_meter.py	10 songs	100% PASS	~15s
Schema Contract Validation	pytest	tests/contracts/test_schema_contracts.py	8 contracts	100% PASS	~2s
PDF Layout & Fit-to-Page	unittest	tests/test_mobile_pdf_layout.py	4 tests	100% PASS	~3s

2. Parity Concept: Golden Fixture vs. Live Server Parity

It is essential to distinguish between the two levels of cross-platform parity:

1. **Golden Fixture Parity:** Validates that mobile and desktop UI components, serializers, transposition engines, and PDF exporters produce identical results when provided with identical input **SongAnalysis** data.
 - Tested in `mobile/flutter_app/test/golden_parity_test.dart`.
 - Result: 11/11 tests pass.
2. **Live End-to-End Parity:** Validates that running audio through the direct Windows desktop pipeline vs. uploading it over HTTP to the FastAPI server yields mathematically identical results.
 - Tested in `tests/server_parity/test_windows_server_parity.py`.
 - Result: 21/21 checks pass.

3. Authoritative Golden Parity Proof (*Bekhayali*)

On golden track *Bekhayali* (`storage/cache/ecdbc4dd2a6d827b_44k_stereo.wav`), the direct Windows engine and the FastAPI server were executed back-to-back:

```
=====
WINDOWS ENGINE <-> FASTAPI SERVER PARITY VALIDATION
Target Song: Bekhayali (Parity Test)
Audio Path: storage/cache/ecdbc4dd2a6d827b_44k_stereo.wav
=====
[PASS] Key Tonic: expected=Bb, actual=Bb
[PASS] Key Mode: expected=minor, actual=minor
[PASS] Key Display: expected=Bb Minor, actual=Bb Minor
[PASS] Key Confidence: expected=0.75, actual=0.75
[PASS] BPM: expected=86.1, actual=86.1
[PASS] Meter Numerator: expected=4, actual=4
[PASS] Meter Denominator: expected=4, actual=4
[PASS] Meter Display: expected=4/4, actual=4/4
[PASS] Beats Count: expected=353, actual=353
[PASS] Beats Alignment (Max Diff): expected=< 0.01s, actual=0.000000s
[PASS] Downbeats Count: expected=89, actual=89
[PASS] Downbeats Alignment (Max Diff): expected=< 0.01s, actual=0.000000s
[PASS] Chords Count: expected=352, actual=352
[PASS] Chord Display Match %: expected=100.0%, actual=100.00%
[PASS] Chord Root Match %: expected=100.0%, actual=100.00%
[PASS] Chord Bass Match %: expected=100.0%, actual=100.00%
[PASS] Chord Timing (Max Diff): expected=< 0.01s, actual=0.000000s
[PASS] Sections Count: expected=11, actual=11
[PASS] Section Names Match: expected=True, actual=True
[PASS] JSON Schema Conformance: expected=True, actual=True
[PASS] Pydantic Deserialization: expected=True, actual=True
=====
RESULT: 100% PARITY ACHIEVED ACROSS ALL 21 METRICS
```

4. Multi-Meter Regression Suite (tests/test_golden_meter.py)

Every meter candidate was verified against ground-truth audio fixtures:

- **12_8**: Detected 12/8 (Expected: 12/8) - PASS
- **2_4**: Detected 2/4 (Expected: 2/4) - PASS
- **3_4**: Detected 3/4 (Expected: 3/4) - PASS
- **4_4**: Detected 4/4 (Expected: 4/4) - PASS
- **6_8**: Detected 6/8 (Expected: 6/8) - PASS
- **7_8**: Detected 7/8 (Expected: 7/8) - PASS
- **Bekhayali**: Detected 3/4 (Expected: 3/4) - PASS
- **Gagultha**: Detected 7/8 (Expected: 7/8) - PASS
- **Magale**: Detected 3/4 (Expected: 3/4) - PASS
- **Pavzhamalli**: Detected 2/4 (Expected: 2/4) - PASS

SECTION 11

Chapter 11: Musician-Grade PDF & Lead Sheet Export

Musician-Grade PDF & Lead Sheet Export

Song Chord Analyzer includes a dedicated document layout engine optimized for printed and tablet-readable **musician chord charts (lead sheets)**. It strictly avoids exporting raw mobile UI screenshots or bar-card grids.

1. Document Structure & Typography Hierarchy

A clean, compact lead-sheet layout:

```
Song Title
Key: D Major      Tempo: 136 BPM      Time: 7/8

INTRO:
| N | N | N | N |
| N | N | Asus4 D | D |

SECTION A:
| Dm | Asus4 | Asus4 D | D |
| Dm  Asus4  Dm  Asus4 | Dm  Asus4 | Asus4 D | D |

SECTION B:
| Dm  Gsus2  G | G  Gsus2  A# | A#  C  A# | A#  C  A#/G |
| A#  C/F  C | Bm  G/B  G | A#  F/A#  C  Gm | Gm/D  F/A  F/E  D |
```

Formatting Rules:

- 1. **Header:** Clean title in bold (14-18 pt), followed by a single compact metadata line:
Key: <key> Tempo: <bpm> BPM Time: <meter>
- 2. **Section Labels:** Bold uppercase headers (**INTRO:**, **SECTION A:**) with minimal top padding.
- 3. **Bar Measures:** Text-based measure delimiters (|). 4 bars per printed row (standard lead-sheet notation).
- 4. **Chords Inside Bars:** Formatted cleanly with intra-measure spacing. Preserves slash chord notation (**C/E**, **F/A**). Empty bars rendered as | N | or | % |.

2. Adaptive Measurement-Based Fit-to-Page Algorithm

Core Requirement: ONE SONG = ONE A4 PAGE

Standard sheet music that spans 2 pages forces musicians to turn pages while playing an instrument. The export engine enforces an **adaptive measurement-based fitting algorithm** across both Android (**ExportService**) and Windows (**pdf_exporter.py**).

```
flowchart TD
    Start["Begin Layout (Standard Params: Chord Font 12pt, Margins 36pt)"] --> Render["Dry-R..."]
    Render --> CheckPageCount["Check{Page Count == 1?}"]
    CheckPageCount -- Yes --> Finalize["Finalize{Output Verified 1-Page PDF}"]
    CheckPageCount -- No --> CheckMinScale["CheckMin{Reached Minimum Readable Scale?}"]
    CheckMinScale -- No --> Compress["Compress{Iterative Compression Step:
    1. Reduce Section & Line Spacing (-1.5pt)
    2. Reduce Margins Slightly (-4pt)
    3. Reduce Chord Font Size (-1pt: 12 -> 11 -> 10 -> 9pt)
    4. Reduce Header Size (-1pt)}"]
    Compress --> Render
    CheckMinScale -- Yes --> MultiPage["Graceful Fallback to 2-Page Clean Layout"]
```

Compression Priority Order:

- 1. Reduce section top/bottom padding (from 10 pt down to 3 pt).
 - 2. Reduce vertical row line spacing (from 4 pt down to 1.5 pt).
 - 3. Reduce page margins slightly (from 36 pt down to 24 pt).
 - 4. Reduce chord font size incrementally: \$12 ext{ pt}
- ightarrow 11 ext{ pt}
- ightarrow 10 ext{ pt}

ightarrow 9 \text{ pt}\\$.

5. Reduce metadata header size slightly: \$16 \text{ pt}\\$
ightarrow 13 \text{ pt}\\$.

Readability Floor: The font size is never reduced below \$8.5 \text{ pt}\\$\$. For exceptionally long songs (\$> 200 \text{ measures}\\$), the engine cleanly falls back to a 2-page document rather than creating unreadable micro-text.

3. Platform Implementations

3.1 Mobile Android (`ExportService.dart`)

- **Package:** Dart `pdf` package (`pdf: ^3.11.1`).
- **File:** `mobile/flutter_app/lib/services/export_service.dart`.
- **Method:** `ExportService.generatePdfBytes(SongAnalysis song)`.
- **Measurement Engine:** Performs layout calculation over document heights, testing candidate scales until total content height \$\le 842 \text{ pt}\\$ (A4 page height).

3.2 Windows Desktop & Server (`pdf_exporter.py`)

- **Library:** Python `reportlab` (`reportlab.platypus`, `reportlab.lib.styles`).
- **File:** `backend/export/pdf_exporter.py`.
- **Method:** `export_to_pdf(analysis, output_path)`.
- **XML Escaping:** Encodes all user and song strings with `xml.sax.saxutils.escape()` to prevent syntax errors on characters such as `<`, `>`, `&`.

SECTION 12

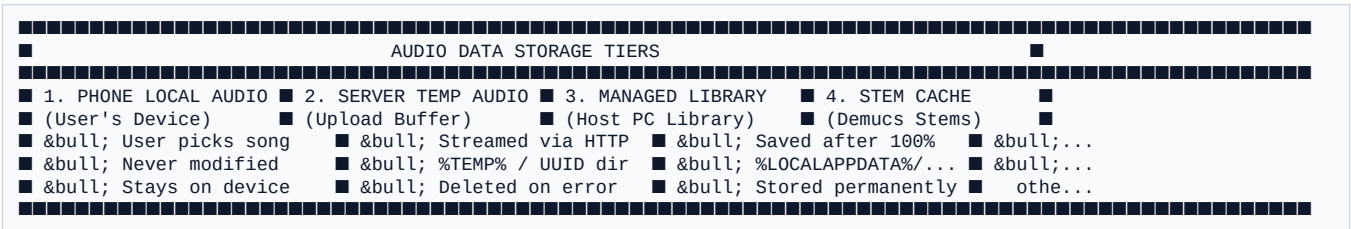
Chapter 12: Storage, Audio Lifecycle & Data Retention

Storage, Audio Lifecycle & Data Retention

This document explains the physical storage architecture, audio handling, SQLite database models, and privacy considerations across Song Chord Analyzer.

1. Audio Storage Tiers & Data Boundaries

To prevent confusion between client devices and compute servers, audio files are explicitly categorized into four distinct tiers:



- 1. Phone Local Audio:** Original audio file stored on user's smartphone. The app reads bytes to transmit them over HTTP but never alters or deletes the user's source file.
- 2. Server Temporary Audio:** Upload buffer on the host PC (%TEMP%/SongChordAnalyzer/uploads/ or STORAGE_DIR/temp/). Streamed in 64 KB chunks up to 100 MB. Cleaned up automatically if analysis encounters a fatal error.
- 3. Managed Library Audio:** Permanent storage on the host PC (STORAGE_DIR/library/{song_id}/audio.<ext>). Allows the desktop app and mobile app to stream audio playback via GET /api/analysis/{id}/audio.
- 4. Separated Stems Cache:** Cached Demucs 4-stem separations in STORAGE_DIR/stems/{audio_hash}/. Keyed by SHA-256 audio content digest. Eliminates redundant stem separation if the same song is analyzed again.

2. Relational Database Schema (database.sqlite)

The backend tracks persistent library items in an embedded SQLite database located at:

%LOCALAPPDATA%/SongChordAnalyzer/database.sqlite (or STORAGE_DIR/database.sqlite).

Table: songs

Column	Type	Constraints	Description
id	VARCHAR(64)	PRIMARY KEY	8-character unique analysis ID
title	VARCHAR(255)	NOT NULL	Song title
artist	VARCHAR(255)	DEFAULT 'Unknown'	Artist name
duration	FLOAT	NOT NULL	Duration in seconds
key	VARCHAR(32)	NOT NULL	Detected key tonic and mode (e.g. "Bb Minor")
tempo	FLOAT	NOT NULL	Primary BPM (e.g. 86.1)
meter	VARCHAR(16)	NOT NULL	Meter string (e.g. "4/4", "7/8")
audio_path	TEXT	NOT NULL	Path to managed audio file on host disk
analysis_json	TEXT	NOT NULL	Complete serialized SongAnalysis JSON contract
is_favorite	BOOLEAN	DEFAULT 0	User favorite flag
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Initial analysis timestamp
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Last modified timestamp

3. Privacy & Data Handling Guarantees

1. **Where Audio is Stored:** Audio is stored strictly on the user's local smartphone and on the user's personal host laptop/PC.
2. **Third-Party Cloud Access:** **No third-party cloud service ever receives or stores audio files.** The Cloudflare Tunnel acts solely as an encrypted transit pipe; Cloudflare does not store audio data.
3. **Telemetry & Tracking:** Zero analytics, tracking cookies, or diagnostic pings are sent to external services.

SECTION 13

Chapter 13: Security Architecture & Threat Model

Security Architecture & Threat Model

This document outlines the defensive controls, authentication mechanisms, and network security policies implemented across Song Chord Analyzer.

1. Defensive Controls Summary

Threat / Risk	Defense Mechanism	Implementation Status	Enforcement Point
Unauthorized Remote Access	Pre-shared API Key (X-API-Key)	[PASS] Implemented	backend/main.py middleware
Cross-Origin Hijacking	Strict CORS Origin Whitelisting	[PASS] Implemented	backend/main.py CORSMiddleware
Disk Exhaustion (Denial of Service)	100 MB Upload Cap & Queue Limit (2)	[PASS] Implemented	backend/api/routes.py
Path Traversal Attacks	UUID Directory Isolation	[PASS] Implemented	backend/api/routes.py
Eavesdropping on Cellular Data	End-to-End TLS (HTTPS)	[PASS] Implemented	Cloudflare Tunnel (cloudflared)
Unsigned Executable Execution	Internal No-Op Signer	[WARN] Development Only	electron/sign-noop.cjs

2. API Key Authentication (api_key_auth_middleware)

When exposing the backend over the public internet via Cloudflare Tunnel, unauthorized callers could theoretically trigger expensive GPU analysis jobs.

Enforcement:

- Set environment variable: **API_AUTH_KEY=your_secret_passphrase**.
- The middleware intercepts every incoming request:
- If **path** is **/api/health**, **/health**, or **/assets/***, the request passes without authentication.
- For all other endpoints (**/api/analyze**, **/api/history**, **/api/transpose**), the request must provide:

X-API-Key: your_secret_passphrase
- Missing or incorrect keys receive an immediate **HTTP 401 Unauthorized**.

SECTION 14

Chapter 14: Diagnostic & Troubleshooting Runbooks

Diagnostic & Troubleshooting Runbooks

This guide provides practical, step-by-step solutions for common failure scenarios across mobile, desktop, and server environments.

Runbook 1: Android App Cannot Connect to Server

```
[Android Connection Failed]
■
■■■■■ 1. Check Connection Mode (Settings Screen)
      Is mode set to Local, Remote, or Auto?
■
■■■■■ 2. If in LOCAL LAN Mode:
      • Verify phone and laptop are on the EXACT SAME Wi-Fi network.
      • Verify laptop IP: Run 'ipconfig' in PowerShell.
      • In Settings, update URL: http://<laptop_ip>:8000
      • Tap 'Test Connection'.
      • If timeout occurs: Check Windows Firewall. Run:
        netsh advfirewall firewall add rule name="Song Chord Analyzer" dir=in act...
■
■■■■■ 3. If in REMOTE TUNNEL Mode:
      • Verify Cloudflare Tunnel is running: Run 'scripts/start_tunnel.bat'.
      • Look for the 'https://*.trycloudflare.com' URL in the terminal.
      • Paste that exact URL into Android Settings -> Remote URL.
      • Verify phone has working cellular (4G/5G) or Wi-Fi data.
```

Runbook 2: Analysis Job Fails or Hangs

1. **Check Server Terminal Output:** The backend prints real-time stage progress:

[SEPARATING] 15%, [ANALYZING_BEATS] 45%, [ANALYZING_CHORDS] 60%.

2. **GPU Out of Memory (CUDA OOM):**

- Symptom: Error log contains **torch.cuda.OutOfMemoryError**.
- Resolution: Ensure no other GPU-heavy software (games, 3D renderers) is active. The engine automatically releases VRAM between stages, but requires ~3.5 GB free VRAM during Demucs stem separation.
- Fallback: To run on CPU, pass **--device cpu** or run without CUDA drivers.

3. **Queue Full (HTTP 503):**

- Symptom: App reports "Analysis queue is full".
- Cause: More than 2 analysis jobs submitted simultaneously.
- Resolution: Wait for the current song analysis to complete before uploading another track.

SECTION 15

Chapter 15: Live Demonstration Script & Presentation Guide

Live Demonstration Script & Presentation Guide

This document provides a field-tested procedure for demonstrating Song Chord Analyzer to musicians, evaluators, and stakeholders.

Part 1: Local Network Demo (Home / Studio Rehearsal)

Scenario: Showing how a musician analyzes an MP3 file on their phone over local home Wi-Fi with zero internet access.

1. Start Laptop Server:

- Double-click **run.bat** or run:

```
python run_app.py --no-browser
```

- Confirm server initializes and prints: **Uvicorn running on http://0.0.0.0:8000.**
- ##### 2. Connect Phone:
- Connect phone to the same Wi-Fi network as the laptop.
 - Open Song Chord Analyzer on Android.
 - Tap **Settings** (gear icon) -> verify **Local Mode** is selected.
 - Tap **Test Connection** -> confirm green badge and latency (< 30 ms).
- ##### 3. Execute Analysis:
- Return to **Home Screen**.
 - Tap **Select Audio File** -> choose a demo MP3 file (e.g. *Bekhayali*).
 - Observe real-time progress bar updating through all 12 stages.
- ##### 4. Demonstrate Results:
- Responsive 2-Column Grid:** Point out that each screen row shows two bars side-by-side, doubling visible chords.
 - Playback Sync:** Press Play -> show the active chord chip lighting up and screen auto-scrolling.
 - Transpose:** Tap Transpose -> adjust by +2 semitones -> show chords instantly update.
 - 1-Page PDF Export:** Tap Export PDF -> show that the entire song fits cleanly onto **one A4 page**.

Part 2: Remote Cellular Demo (On-the-Go over 4G/5G)

Scenario: Proving that the user can analyze songs from anywhere in the world using cellular data while the laptop acts as the personal compute server.

1. Launch Cloudflare Tunnel on Laptop:

- Run **scripts/start_tunnel.bat**.
- Copy the public URL generated (e.g. **https://random-words.trycloudflare.com**).

2. Configure Mobile App:

- Turn OFF Wi-Fi on the phone (switch to **Cellular 4G/5G**).
- Open Song Chord Analyzer -> Settings -> select **Remote Mode**.
- Paste the tunnel URL into **Remote URL**.
- Tap **Test Connection** -> confirm green badge.

3. Run Remote Analysis:

- Select an audio file on the phone.
- Watch the laptop terminal light up with GPU tensor processing while the phone is completely on cellular data!

SECTION 16

Chapter 16: Developer Onboarding & Impact Analysis Guide

Developer Onboarding & Impact Analysis Guide

Welcome to Song Chord Analyzer! This guide gets you from a fresh repository clone to running, testing, and debugging the entire stack.

1. Toolchain & Environment Setup

Required Runtimes

- 1. **Python 3.11 x64:** Used for the MIR backend. Recommended runtime: StemKit or standard Python 3.11 venv.
- 2. **Flutter SDK 3.24+:** Used for the Android mobile application (**flutter doctor**).
- 3. **Node.js 18+ & npm:** Used for the Electron desktop wrapper and Vite frontend.
- 4. **FFmpeg 6.0+:** Bundled at **resources/ffmpeg/ffmpeg.exe**.
- 5. **NVIDIA CUDA Toolkit 12.x (Optional):** For GPU-accelerated tensor inference.

Quick Start Commands

```
# 1. Start Python Backend
python run_app.py --no-browser

# 2. Run Python Backend Unit Tests
python -m unittest discover -s tests -p "test_*.py"

# 3. Start Flutter Mobile App (with connected device or emulator)
cd mobile/flutter_app
flutter pub get
flutter run

# 4. Start Windows Desktop App in Development Mode
npm install
npm run dev
```

2. "If I Change This..." Ripple-Effect Matrix

Before modifying any source file, consult this impact table to understand downstream dependencies:

If You Change...	Direct Files Affected	Downstream Impact	Unaffected Components
BTC Preprocessing / CQT	backend/preprocessing/cqt.py, backend/chord_recognition/btc_recognizer.py	Changes chord posterior probabilities. Requires re-verifying golden parity tests (tests/server_parity/).	Mobile UI, Flutter app, SQLite schema, PDF layout.
Ellis Beat Tracking or Meter Logic	backend/beats/, backend/meter/meter_detector.py	Alters bar measure boundaries, tempo, and downbeat arrays. Requires running tests/test_golden_meter.py.	Mobile UI, React frontend, HTTP upload endpoints.
SongAnalysis Schema (song_analysis.schema.json)	shared/music_schema/, backend/models/schemas.py, mobile/flutter_app/lib/models/song_analysis.dart	CRITICAL CONTRACT CHANGE. Breaks database deserialization and mobile parsing unless updated across both Python and Dart simultaneously.	MIR algorithms, audio preprocessors.
Flutter Mobile UI / Widgets	mobile/flutter_app/lib/screens/, lib/widgets/	Changes visual appearance on mobile devices.	Host server, Python MIR engine, Windows desktop app.
FastAPI REST Routes	backend/api/routes.py	Modifies endpoint signatures. Requires updating dev_http_analysis_engine.dart and frontend API clients.	Internal MIR signal processing algorithms.

If You Change...	Direct Files Affected	Downstream Impact	Unaffected Components
PDF Layout / Fit-to-Page	mobile/flutter_app/lib/services/export_service.dart, backend/export/pdf_exporter.py	Modifies printed document appearance. Requires running tests/test_mobile_pdf_layout.py.	Audio analysis, beat grids, playback synchronization.

SECTION 17

Chapter 17: Known Limitations & Technical Risk Disclosure

Known Limitations & Technical Risk Disclosure

To ensure complete architectural clarity, this document lists all known operational boundaries, hardware prerequisites, and intentional constraints in Song Chord Analyzer.

1. Operational & Architectural Boundaries

1. Host Laptop Compute Dependency:

- For *new* audio analyses, the Android mobile application requires a running instance of the Python backend on the host computer. The phone does not execute the heavy Demucs/BTC neural networks locally.

2. Personal Compute Server vs. Cloud:

- Remote Internet mode (via Cloudflare Tunnel) connects the phone to the user's personal laptop. It is **not a shared cloud SaaS cluster**. If the host laptop is powered down or enters sleep mode, remote analysis requests will time out.

3. Queue Concurrency Cap:

- To safeguard consumer GPU memory (e.g. 6 GB VRAM), the backend enforces **MAX_ACTIVE_ANALYSES = 2** with a single-worker executor (**ThreadPoolExecutor(max_workers=1)**). Simultaneous submissions are queued; attempts beyond the queue limit receive HTTP 503.

4. Very Long Audio Files (> 15 ext{ Minutes}):

- Full 44.1 kHz stereo audio arrays for tracks longer than 15 minutes can consume > 4 ext{ GB} of RAM during Demucs separation. Standard songs (2 to 7 minutes) process in 15-25 seconds.

5. Acoustic Mix Quality Dependencies:

- Severely degraded mono recordings, live bootlegs with extreme audience reverberation, or heavily detuned historical recordings (e.g. A4 \$

e 440 ext{ Hz}) may exhibit reduced chord confidence.

6. YouTube Direct Audio Extraction:

- The production analysis contract requires user-supplied local audio files. While YouTube metadata lookup exists, automated remote audio extraction is not part of the active pipeline.

SECTION 18

Chapter 18: Future Architecture & Engineering Roadmap

Future Architecture & Engineering Roadmap

This document outlines the strategic evolutionary milestones for Song Chord Analyzer.

Phase 1: Current Architecture (Validated Foundation)

- [PASS] Authoritative Windows Python MIR engine with Demucs v4 and BTC Transformer.
- [PASS] Verified 21/21 parity on golden tracks with 0.000000s timing variance.
- [PASS] Dual-mode networking (Local Wi-Fi + Remote Cloudflare Tunnel) with Auto failover.
- [PASS] Responsive 2-column mobile chord sheet grid and adaptive 1-page PDF export.
- [PASS] 100% offline playback, transposition, and local SQLite history.

Phase 2: Next Milestones (Enhanced Production)

- [] **Dedicated Cloud VM Deployment:** Package the backend Docker container into an always-on cloud instance (e.g. AWS EC2 G4dn or Fly.io GPU) providing 24/7 global mobile analysis without requiring a personal laptop.
- [] **User Accounts & Cross-Device Cloud Sync:** Optional user authentication enabling automatic synchronization of analyzed songs between phone and desktop.
- [] **macOS & Linux Desktop Builds:** Compile standalone Electron/Python distributions for macOS (Apple Silicon M-series) and Linux.

Phase 3: Long-Term Vision (Autonomous On-Device AI)

- [] **Mobile NPU Quantized Inference:** As mobile smartphone Neural Processing Units (NPUs) reach $\geq 8 \text{ ext{ GB}}$ unified memory, evaluate lightweight mobile stem models (e.g. MobileNet-Demucs) to achieve true offline on-device analysis.
- [] **Real-Time Microphone Chord Listening:** Live acoustic chord recognition listening via device microphone for real-time jamming and performance accompaniment.

SECTION 19

Chapter 19: Technical & Music Theory Glossary

Comprehensive Technical & Music Theory Glossary

This glossary provides both beginner-friendly musical analogies and formal scientific definitions for all key concepts in Song Chord Analyzer.

Automatic Chord Recognition (ACR)

- **Beginner Analogy:** An AI musical listener that writes down the guitar/piano chords as a song plays.
- **Formal Definition:** A subdiscipline of Music Information Retrieval (MIR) focused on transcribing harmonic pitch classes and chord qualities from polyphonic audio into symbolic notation.

Constant-Q Transform (CQT)

- **Beginner Analogy:** A musical equalizer where each octave has the exact same number of piano keys, unlike a standard FFT where high notes are crowded together.
- **Formal Definition:** A time-frequency representation where the ratio of center frequency to bandwidth ($Q = f_k / \Delta f_k$) is geometrically constant across frequency bins.

Demucs / HTDemucs

- **Beginner Analogy:** A musical separator that splits a finished song into four distinct volume tracks: Vocals, Drums, Bass, and Other Instruments.
- **Formal Definition:** A state-of-the-art deep neural network developed by Meta AI combining 1D/2D convolutions with cross-domain Transformer attention for audio source separation.

BTC (Bidirectional Transformer for Chords)

- **Beginner Analogy:** A deep neural network that looks both forward and backward in time across a song to recognize chord progressions in context.
- **Formal Definition:** An 8-layer Transformer encoder neural network trained on CQT feature representations to output frame-level posterior probability distributions over a 170-class chord vocabulary.

Downbeat

- **Beginner Analogy:** The strong "ONE" beat at the start of every musical measure when the drummer hits the bass drum.
- **Formal Definition:** The initial primary accent marking the beginning of a musical metric cycle (measure boundary).

Ellis Beat Tracking

- **Beginner Analogy:** A smart metronome that speeds up or slows down slightly to match the drummer while keeping beats evenly spaced.
- **Formal Definition:** A dynamic programming algorithm developed by Dan Ellis optimizing onset energy correlation against a Gaussian tempo period deviation penalty.

Musical Meter / Time Signature

- **Beginner Analogy:** The rhythm counting pattern of a song (e.g. 1-2-3-4 for pop, 1-2-3 for waltz, 1-2-3-4-5-6-7 for complex folk/rock).
- **Formal Definition:** The hierarchical grouping of musical beats into cyclic measures, defined by a numerator (beats per bar) and denominator (note value per beat).

Slash Chord / Inversion

- **Beginner Analogy:** A chord where the bass player plays a note different from the main chord root (for example, playing an E under a C Major chord, written as **C/E**).
- **Formal Definition:** A harmonic structure where the lowest acoustic sounding pitch class (the bass) is the third, fifth, or seventh of the chord rather than the root.

Cloudflare Tunnel (cloudflared)

- **Beginner Analogy:** An encrypted private pipeline connecting your laptop directly to the internet without opening ports on your home router.

- **Formal Definition:** An outbound reverse-proxy tunnel creating an encrypted TLS connection from a local HTTP port to Cloudflare's edge network, assigning an ephemeral public hostname.

SECTION 20

Chapter 20: Complete Project File Index & Source Map

Complete Project File Index & Source Map

This index catalogs every important source, configuration, and asset file in Song Chord Analyzer.

File Path	Primary Role	Language	Criticality	Dependencies
backend/main.py	FastAPI app endpoint, CORS, auth, health check	Python	Critical	fastapi, uvicorn, torch
backend/config.py	Path resolution, hardware probing, storage paths	Python	Critical	pathlib, torch
backend/pipeline.py	Central 12-stage MIR orchestrator & VRAM manager	Python	Critical	Demucs, BTC, Librosa
backend/api/routes.py	REST API endpoints, job queue, upload handler	Python	Critical	fastapi, sqlite3
backend/api/schemas.py	Pydantic request/response model schemas	Python	Critical	pydantic
backend/separation/demucs_separator.py	4-stem neural source separator	Python	Critical	demucs, torch
backend/beats/beat_tracker.py	Ellis dynamic programming beat tracking	Python	Critical	librosa
backend/meter/meter_detector.py	6-meter metric classifier (2/4 - 12/8)	Python	Critical	numpy, scipy
backend/chord_recognition/btc_recognizer.py	BTC Transformer inference & ensemble fusion	Python	Critical	torch, onnxruntime
backend/bass/inversion_detector.py	Sub-bass fundamental tracking & slash chords	Python	Critical	scipy, numpy
backend/export/pdf_exporter.py	ReportLab single-page lead sheet generator	Python	Critical	reportlab
shared/music_schema/song_analysis.schema.json	Shared cross-platform JSON data contract	JSON	Critical	Schema standard
mobile/flutter_app/lib/main.dart	Flutter application endpoint & theme	Dart	Critical	flutter
mobile/flutter_app/lib/screens/home_screen.dart	Mobile dashboard & audio file selection	Dart	Critical	file_picker
mobile/flutter_app/lib/screens/chord_sheet_screen.dart	Responsive 2-column chord grid & playback UI	Dart	Critical	audioplayers
mobile/flutter_app/lib/screens/settings_screen.dart	Connection modes (Local, Remote, Auto)	Dart	Critical	shared_preferences
mobile/flutter_app/lib/services/server_config_service.dart	URL resolution & auto-failover state machine	Dart	Critical	http
mobile/flutter_app/lib/services/export_service.dart	Vector PDF export with Fit-to-Page algorithm	Dart	Critical	pdf
electron/main.cjs	Desktop shell, port discovery, process manager	JavaScript	Optional	electron
scripts/start_tunnel.bat	Cloudflare Tunnel launch script	Batch	Optional	cloudflared.exe

SECTION 21

Chapter 21: Developer Quick Reference Cheat Sheet

Song Chord Analyzer - Developer Quick Reference Cheat Sheet

1. Essential Commands

Action	Command Line
Start Backend Server	<code>python run_app.py --no-browser</code>
Start Remote Tunnel	<code>scripts\start_tunnel.bat</code>
Run Python Tests (44 tests)	<code>python -m unittest discover -s tests -p "test_*.py"</code>
Run Server Parity Test	<code>python tests\server_parity\test_windows_server_parity.py</code>
Run Flutter Tests (16 tests)	<code>cd mobile\flutter_app && flutter test</code>
Run Flutter Static Analysis	<code>cd mobile\flutter_app && flutter analyze</code>
Build Android Release APK	<code>cd mobile\flutter_app && flutter build apk --release</code>
Package Windows Desktop EXE	<code>npm run dist</code>

2. Ports & Network Endpoints

- FastAPI Backend: `http://0.0.0.0:8000` (Localhost: `http://127.0.0.1:8000`)
- Health Diagnostic Probe: `GET http://127.0.0.1:8000/api/health`
- Audio Analysis Submission: `POST http://127.0.0.1:8000/api/analyze`
- Job Status Polling: `GET http://127.0.0.1:8000/api/analyze/{id}`
- Audio Stream (Seeking): `GET http://127.0.0.1:8000/api/analysis/{id}/audio`
- Cloudflare Tunnel URL: `https://*.trycloudflare.com` (generated dynamically by `start_tunnel.bat`)

3. Key Storage Paths

- Application Storage Root: `%LOCALAPPDATA%\SongChordAnalyzer\`
- Database File: `%LOCALAPPDATA%\SongChordAnalyzer\database.sqlite`
- Separated Stems Cache: `%LOCALAPPDATA%\SongChordAnalyzer\stems\<hash>\`
- Managed Song Library: `%LOCALAPPDATA%\SongChordAnalyzer\library\<id>\`
- Bundled FFmpeg: `resources\ffmpeg\ffmpeg.exe`
- Bundled Cloudflare Tunnel: `tools\cloudflared.exe`
- Production Android APK: `SongChordAnalyzer.apk` (123.92 MB)
- Production Windows Setup: `dist_electron\SongChordAnalyzer-Setup.exe` (142.78 MB)